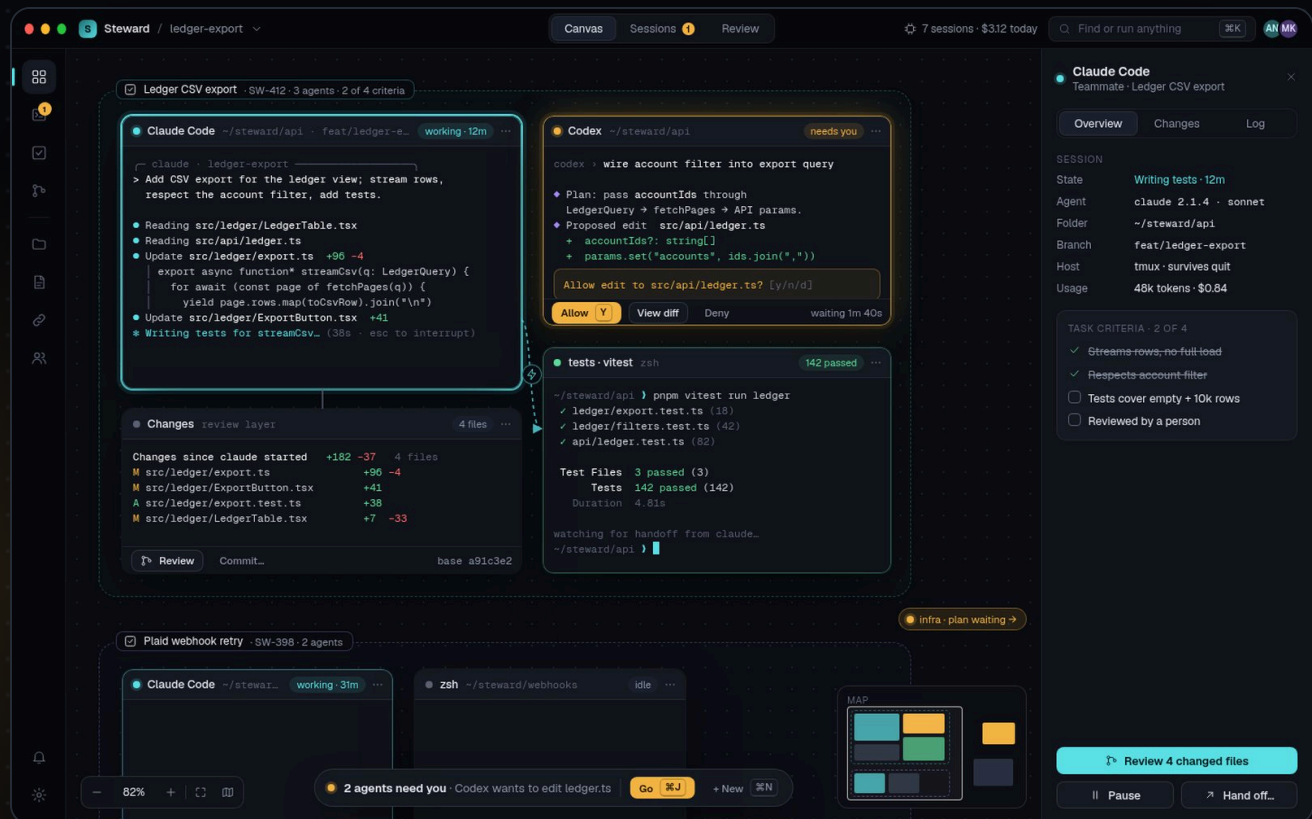


All I Need to Know: the redesign, built by Claude Code

An ordered implementation guide for **alexnieves-cs/terminal-canvas**: foundations first, then screens 01–09, then the 3D World revamp (10–14). It is written to be handed to parallel Claude Code sessions as is.

● working
 ● needs you
 ● done
 ● failed
 ● idle



Quick start and checklist

This page is the whole run in order. Each item points to a section with the details and the prompts to paste. **SEQ** means one session, one step at a time. **PAR** means parallel worktree lanes. **GATE** is an automated check. **YOU** means a person has to look.

0 Lock decisions **YOU**

- Answer D1–D8 (§2.3): cyan selection vs. cyan working, the nav change, metrics in Sessions, ⌘ chord migrations, World in production, version number.
- Write the answers into `docs/redesign/DECISIONS.md` before any lane starts.

1 Set up the machine **SEQ**

- macOS, Node 20, tmux, Xcode CLT, gh. Update Claude Code (`claude update`).
- Clean clone, `npm ci`, then `node node_modules/electron/install.js`.
- Baseline: `npm run verify` and `npm run shot`. Save timings and failures.

2 Phase 0: the run kit **SEQ** **GATE**

- One lead session in plan mode creates `redesign/main`, the mockups in `docs/redesign/`, `ownership.json`, lane briefs, `.claude/` agents, commands and hooks.
- Pre-cut seams: CSS sections, shot scenes, verify scripts, contract types.
- Exit: verify green, tag `rd-p0`.

3 Phase 1: foundations **PAR** ×3

- F1 tokens and the state colour system · F2 shared model (attention queue, task regions, zoom tiers, canvas↔world space) · F3 shell frame (nav, shortcut registry, focus lock, pill).
- Merge F1 → F2 → F3. Tag `rd-foundations`. Freeze the shared files.

4 Phase 2: canvas screens 01–09 **PAR** ×4 then ×2

- Wave 2a: L–A launch and first run (01–03) · L–B workspace and arrange (04–05) · L–D Sessions (07) · L–E Settings keyboard (08).
- Wave 2b, after L–B merges: L–C navigate, tiers and palette (06) · L–F errors and recovery (09).

5 Phase 3: 3D World 10–14 **W0** **PAR** ×3, ×2, ×1

- W0 night studio and one colour system (sequential).
- Then W1 transition (10) || W2 room (11) || W3 overview, camera and time (14).
- Then W4 focus (12) || W6 fallback and doors. Last, W5 attention flight (13).

6 Gates on every lane **GATE** **YOU**

- G1 on each stop: typecheck, `npm run affected`, and the lane's own checks.
- G2 before merge: full `npm run verify` (behind the Electron lock), `npm run shot + .vs-reference.png`, mockup-critic and rules-reviewer verdicts.
- G3 after merge, on the integration branch: verify again.

7 Phases 4–5: integrate and ship **SEQ** **YOU**

- Run the critic on all 14 scenes, write ledger sentences, then `UPDATE_GOLDENS=1` only after a person has looked.
- Manual QA (§9.4), `verify:packaged`, version bump, `npm run package`, merge to `main`, tag.

THE FIVE RULES THAT PREVENT MOST OF THE PAIN

1) Foundations merge before any screen lane starts. 2) Every file has exactly one owning lane per wave (`docs/redesign/ownership.json`), and a hook enforces it. 3) Shared append points are pre-cut in Phase 0, so lanes never edit the same lines. 4) Electron-tier verification runs one lane at a time behind a lock, because the harness uses fixed fixture paths. 5) A golden changes only after the critic's sentence is in the ledger and a person has looked.

Table of contents

Quick start and checklist	00
What you are building, and what the repo already has	5
1.1 The repo at a glance (read from GitHub, main branch)	5
1.2 Where each redesign idea lands in today's code	6
1.3 House rules every agent must keep (from CLAUDE.md and product-rules.md)	6
The orchestration model	8
2.1 Roles	8
2.2 Phases, waves and agent counts	9
2.3 Decisions to lock before kickoff (D1–D8)	9
2.4 Worktrees, branches and merging	10
2.5 How conflicts are avoided	10
2.6 Verification gates	11
Prerequisites and machine setup	12
3.1 Tools	12
3.2 Baseline (do this before Claude touches anything)	12
3.3 Put the spec inside the repo	12
The Claude Code kit (created in Phase 0)	14
4.1 CLAUDE.md: the redesign section	14
4.2 Subagents (.claude/agents/)	14
4.3 Slash commands (.claude/commands/)	17
4.4 Hooks and permissions (.claude/settings.json)	18
4.5 Run scripts (scripts/redesign/)	19
4.6 Headless mode, where it pays off	20
Run kit, seams and contracts	21
Foundations	24
F1 · Design tokens and the one state colour system	24
F2 · The shared model: attention queue, regions, zoom tiers, canvas↔world space	26
F3 · The shell frame: nav, shortcut registry, focus lock, pill	27
Canvas screens 01–09	29
Lane L-A · 01 Launch/restore, 02 Onboarding, 03 Empty canvas	29
Lane L-B · 04 Main workspace, then 05 Create and arrange	32
Lane L-D · 07 Sessions (multi-session management)	35
Lane L-E · 08 Settings › Keyboard	36
Wave 2b · launch	38
Lane L-C · 06 Navigate: Plan tier and the ⌘K palette	38
Lane L-F · 09 Errors and disconnected	39

The 3D World revamp, 10–14 42

 W0 · Night studio and one colour system in the room 42

 Wave 3b · launch 43

 W1 · 10 Canvas → World transition 43

 W2 · 11 The World, Plan tier 45

 W3 · 14 Overview, camera and time 46

 Wave 3c · launch 48

 W4 · 12 Focus and act 48

 W6 · WebGL fallback and the doors both ways 49

 W5 · 13 Attention flight (ꞤJ) 49

Integration, testing and release 51

 9.1 Phase 4: integration and goldens (M455) 51

 9.2 What proves what 51

 9.3 Performance and robustness checks 52

 9.4 Manual QA checklist 52

 9.5 Phase 5: release (M456) 53

Risks and gotchas 55

Lane cheat sheet 57

Claude Code features this plan relies on 58

What you are building, and what the repo already has

The redesign is "a calm canvas where only state is loud." The chrome steps back. One state colour system shows the same way on every surface. Tasks become visible territories, and the canvas gets three semantic zoom tiers (Work, Plan, Map). The 3D World becomes "the canvas, stood up": the same layout, the same colours and the same attention queue, seen through a 3D lens.

1.1 The repo at a glance (read from GitHub, main branch)

Fact		What it means for this run	
Electron 43 · electron-vite 2 · React 19 · TypeScript 5.7 · xterm 5.5 (+ webgl addon) · node-pty 1.1 · zustand · Radix · motion · three 0.186 · @react-three/fiber 9.8 · drei 10.7 · postprocessing		No new styling dependency is allowed (no Tailwind, no component library, no icon font). <code>icons.tsx</code> grows instead. Every library is reached through one door module (<code>src/rendererer/CLAUDE.md</code>).	
<code>npm run verify = scripts/verify-all.cjs</code>		This is the only test story: no unit-test runner and no linter. The suite list is derived from the <code>verify:*</code> keys in package.json, so a new suite runs just by existing. It runs in three waves: the plain-node suites concurrently, then <code>npm run build</code> , then the Electron suites serially.	
<code>npm run affected, npm run lb -- <module></code>		<code>affected</code> runs only the suites your diff reaches, which makes it a fast inner-loop gate. <code>lb</code> searches <code>docs/load-bearing.md</code> . Run it before you change a module.	
<code>npm run shot · verify:visual · UPDATE_GOLDENS=1</code>		The shot script captures 23+ real-renderer PNGs. Goldens live in <code>verify/visual/goldens/</code> (79 files) at 1440×865. A scene can name a <code>reference</code> PNG, and the shot script then writes <code><scene>.vs-reference.png</code> composites for a fresh-context critic.	
<code>.github/workflows/verify.yml</code>		CI is macOS only (native node-pty + tmux) and runs exactly <code>npm run verify</code> .	
CLAUDE.md files: root, <code>src/main/</code> , <code>src/rendererer/</code> , <code>src/rendererer/canvas/</code>		The repo already uses nested CLAUDE.md memory. This run adds a redesign section and a <code>.claude/</code> kit. There is no <code>.claude/</code> directory today.	
Build logs, numbered up to M432–M434 (world cross-link)		This run takes M435–M456 and one ledger, <code>docs/build-log/m435-m456-ledger.md</code> . <code>verify:meta ledger.1</code> goes red if the ledger isn't linked from the root CLAUDE.md.	

1.2 Where each redesign idea lands in today's code

Redesign idea	Existing seam (real paths)	Gap to close
One state colour system	<code>src/renderer/panels/panel-state.ts</code> (THE one vocabulary: closed Tone set), <code>[data-tone]</code> rules in <code>src/renderer/styles.css</code> , <code>primitives/StatusDot.tsx</code> , <code>shell/chart-tokens.ts</code>	Today working = blue (<code>--blue</code>), idle = green , done = muted . The concept wants cyan working, green done and slate idle. The World uses its own hexes (<code>world/world-palette.ts</code> STUDIO, candy robot tints).
Task regions	<code>canvas/task-clusters.ts</code> , <code>TaskClusterLayer.tsx</code> , <code>task-members.ts</code> , <code>shared/task-plan.ts</code> , <code>shared/checklist.ts</code>	The region becomes a labelled dashed territory with ticket, agent count and criteria progress. ⌘G makes a task from a selection.
Semantic zoom Work/Plan/Map	<code>canvas/lod.ts</code> (live/card terminal tiering : LIVE_BUDGET 8, LIVE_MIN_SCALE 0.5), <code>tier-fade.ts</code> , <code>card-detail.ts</code> , <code>flight.ts</code> , <code>viewport.ts</code>	Add a new semantic-tier layer on top of <code>lod.ts</code> . The WebGL budget and dormancy tiers are listed as untouchable in product-rules.
One attention story, ⌘J	<code>canvas/attention.ts</code> , <code>shared/attention-words.ts</code> , <code>shell/decision-inbox.ts</code> , <code>task-queue.ts</code> , <code>CommandPill.tsx</code> , <code>EdgeIndicators.tsx</code> , <code>MinimapOverlay.tsx</code> , <code>attention:jump</code> IPC	One ordered queue selector that every surface reads, World included.
Canvas Sessions Review nav	<code>shell/TopBar.tsx</code> SegmentedControl over <code>CenterView</code> (<code>canvas orchestration team</code>) from <code>useShellChrome.ts</code>	Sessions and Review become center views. Orchestrate (M268) needs a new home (decision D2).
Shortcuts, ⌘-gated	<code>canvas/useKeyboardNav.ts</code> (⌘arrows, ⌘⇧, ⌘Esc, ⌘⇧Space), <code>main/menu.ts</code> (⌘⇧N, ⌘⇧T tidy, ⌘⇧F flip), palette ⌘K, broadcast ⌘⇧	There is no single shortcut registry, and Settings > Keyboard (O8) needs one. The concept moves Tidy to ⌘⇧T.
3D World	<code>src/renderer/world/*</code> (40 files), <code>world-toggle.ts</code> (one bit, <code>#/world</code>), <code>world-transition.ts</code> (pure 1000 ms clock), <code>WorldStage.tsx</code> (the one <code>lazy()</code>), <code>WorldFlat.tsx</code> , <code>webgl-probe.ts</code> , <code>main/world-feed-link.ts</code> , <code>main/world-sim.ts</code> , <code>scripts/verify-world.cjs</code>	Night studio, state on robots, terraces at canvas coordinates (px ÷ 100), agents stay at their task, ⌘⇧W, the shared chrome, focus sheet, flight. There are no World goldens yet.

1.3 House rules every agent must keep (from CLAUDE.md and product-rules.md)

- **Restyle, don't rename.** About 200 checks select on the DOM aliases (`.panel__*`, `*-node__*`). Never
- **The metrics rule.** CPU, tokens and \$ belong in the inspector, never in headers, cards, the rail or the

transform `.pf__body`.

- **Chromeless chrome is absolutely positioned.** A collapsing hover box refits xterm and sends a SIGWINCH to the agent (measured in M234).
- **The state vocabulary lives in panel-state.ts only.** `verify:rail state.2` greps for state literals everywhere else.
- **The face rule.** Mono is for code, paths and cells only. Sentences use `--font-ui` (already Geist-first).
- **The rest rule.** At rest a surface shows a glyph, a name and one state. Verbs appear at opacity 0 → 1 in `--dur-1`.

status bar (see D3).

- **Words, not codes.** Empty states go through `shell/EmptyState.tsx` + `shared/empty-states.ts`. Never show a bare zero.
- **Tokens.** A new token name is declared in **both** theme blocks (`verify:styles theme.1`). Re-valuing an existing token is recorded in the ledger.
- **Checks use scoped ids** (`ok('rd-tone.1 ...')`). Never take the next global integer (`verify:meta 22`).
- **Library doors.** three, R3F and drei only inside the listed world files, behind `WorldStage`'s `lazy()`. No drei `<Environment>` (the CSP blocks the CDN).

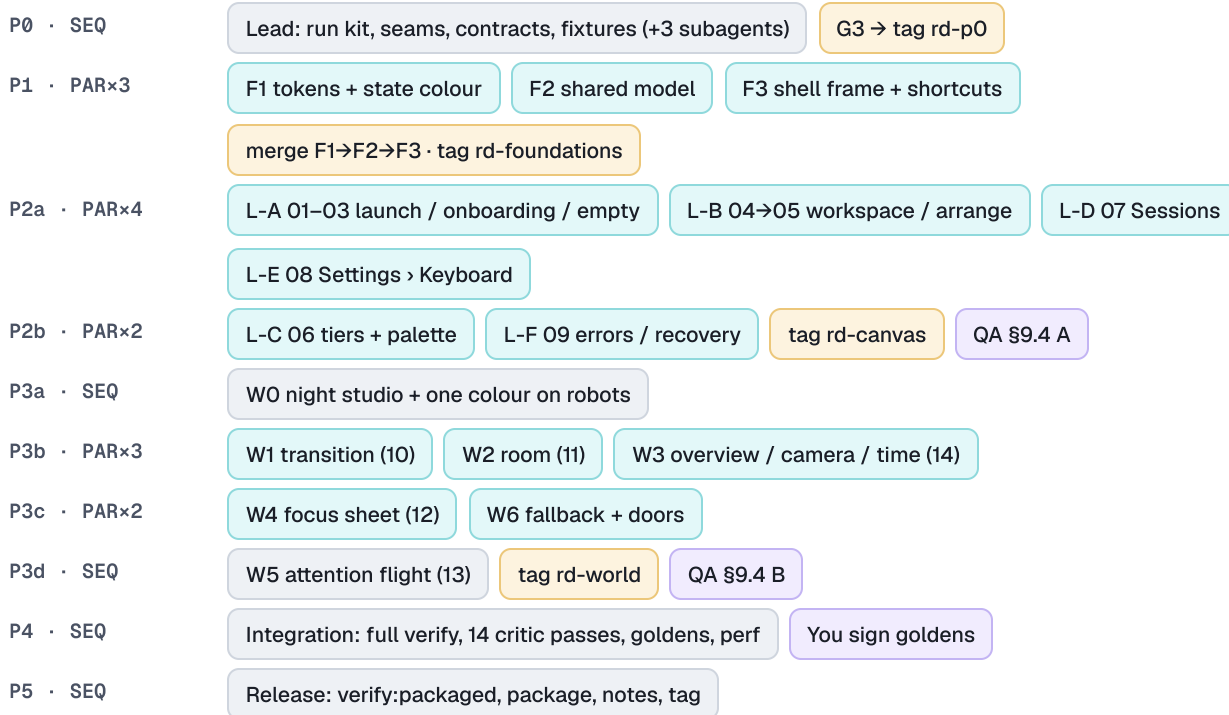
The orchestration model

Use Claude Code at full width: a **lead session** that integrates, several **lane sessions** working in parallel (one git worktree each), and **subagents** inside every session for scouting, verification and critique. The plan is designed for parallel work, but its order is strict. Shared foundations land first, every file has one owner per wave, and nothing merges without passing its gates.

2.1 Roles

Role	Where it runs	What it does
Lead (1 session)	Main checkout on <code>redesign/main</code>	Runs Phase 0. Owns the hotspot files (<code>Canvas.tsx</code> seams, <code>WorldView.tsx</code> wiring, <code>package.json</code> , <code>ipc-contract.ts</code>) between waves. Rebases and merges lanes in a fixed order. Runs G3. Writes the ledger. Never implements screen features.
Lane (1–4 at a time)	A worktree at <code>../tc-rd-<lane></code> on branch <code>rd/<lane></code> , each in its own tmux window	Implements one lane brief. Starts in plan mode, gets the plan approved, then implements in milestone-sized commits. Runs G1 continuously and G2 before asking to merge. Edits only the files that ownership.json gives it.
Subagents (per session)	<code>.claude/agents/*.md</code>	<code>lb-scout</code> (read-only constraints), <code>check-author</code> (verify checks written in the repo's idiom), <code>verify-runner</code> (runs gates and returns only failures), <code>rules-reviewer</code> , <code>world-guard</code> , and <code>mockup-critic</code> (fresh context: it judges the capture against the mockup).
Headless jobs	<code>claude -p ... --output-format json</code>	Batch critic runs over every <code>.vs-reference.png</code> , plus nightly status digests. These don't need a person until they report.
You	—	Approve plans, look at composites, sign off on goldens, and run the manual QA.

2.2 Phases, waves and agent counts



Why this shape. Lanes in the same wave own disjoint file sets, so a merge is mechanical. Waves are cut where real dependencies are. L-C's tiers and L-F's recovery states render the panel frame and regions that L-B builds. W4 and W5 need the room from W2. W5's flight needs W4's focus landing. A wave can have at most four lanes because the Electron suites run serially. More lanes than that just queue for the verification lock.

2.3 Decisions to lock before kickoff (D1–D8)

These are real conflicts between the concept and the repo's current rules. Answer them in `docs/ redesign/ DECISIONS.md`. Every lane brief points to that file. The defaults are my recommendations.

#	Conflict	Recommended default
D1	Cyan selection vs. cyan working. M63/M279 rule: "selection is cyan; work is blue." The concept makes cyan both the accent and the working state, and draws selection as a 2px cyan ring.	Accept cyan for working. Selection becomes a 2px ring + 6px halo in --fg - tinted cyan at higher luminance (#A6F6FF) plus the inspector binding, so a selected working panel still reads as two facts. Record the rule change in the ledger and in product-rules.md.
D2	Nav. Today: Canvas Orchestrate People. The concept: Canvas Sessions Review. It also says the Orchestration view (M268) is kept.	The center control becomes Canvas Sessions Review . Orchestrate moves to the View menu, the palette ("Show Orchestrate") and a Sessions header link. People stays conditional. World stays a lens inside Canvas, not a tab.
D3	Metrics rule vs. mockups. The Sessions table shows Run and Cost columns. The title bar shows "7 sessions · \$3.12 today". The inspector shows usage \$0.84 (allowed).	Sessions is a triage view at the <i>inspector/deep-detail</i> density layers, so its columns are allowed. Amend <code>metrics.1</code> to scope to canvas surfaces. Drop "\$3.12 today" from the title bar and keep "7 sessions". The spend total lives in the Sessions header.

#	Conflict	Recommended default
D4	⌘K clash. Claude Code's TUI (and macOS terminals) use ⌘K to clear the screen.	The canvas owns ⌘K unless the focused panel is in focus lock (⌘⇧L) , in which case every key goes to the terminal. This is shown in 08.
D5	Chord migrations. Tidy is ⌘⇧T today and ⌘⇧T in the concept. New chords: ⌘1/2/3, ⌘O, ⌘⇧O, ⌘G, ⌘Y, ⌘., ⌘L, ⌘⇧S, ⌘⇧W, ⌘⇧L, ⌘⇧J.	Keep the old chord as a hidden alias for one release. Every new chord is registered in <code>src/shared/shortcuts.ts</code> , and a check fails on duplicates.
D6	World in production. A TopBar comment says the toggle is dev-only. renderer/CLAUDE.md says Canvas mounts it in production since M427.	Ship it on by default with the WebGL probe and WorldFlat fallback. Verify in a packaged build (<code>verify:packaged</code>) before release.
D7	Tone remap side effects. Watcher "passed", work "done" and auto "done" map to <code>idle</code> (green) today.	Move "finished OK" facts to the <code>done</code> tone (green). <code>idle</code> becomes slate. Words stay the same. Only tones move.
D8	Version.	Ship as 6.0.0 (the visual language changes) with a migration note on chords.

2.4 Worktrees, branches and merging

- **Integration branch:** `redesign/main`, cut from `main` at tag `pre-redesign`. `main` stays releasable the whole time.
- **Lane branches:** `rd/<lane>` (for example `rd/l-b-workspace`), always cut from the *current* `redesign/main` tag for the wave. Create them with plain `git worktree add`, not `claude --worktree`. Claude Code's `--worktree` flag (and `isolation: worktree` subagents) branch from the repository's **default branch** unless `worktree.baseRef` is set to `"head"`. We want `redesign/main`, and explicit names. Keep `isolation: worktree` for disposable subagent experiments, with `worktree.baseRef: "head"` in `.claude/settings.json`.
- **Each worktree needs its own `npm ci`.** `postinstall` runs `electron-rebuild` against `node-pty`. Don't symlink `node_modules` between worktrees.
- **Merge protocol (lead only):** the lane rebases onto `redesign/main`, runs G2, and opens a PR (`gh pr create -b redesign/main`). The lead merges with `--no-ff` in the wave's fixed order, runs G3, and tags. Lanes still running in the wave then rebase. Squash only at the very end (or never: milestone commits are the history).
- **Commit style:** the repo convention, `feat(m442): ...` / `fix(m442): ...`, so the history maps to the ledger.

2.5 How conflicts are avoided

Mechanism	How it works
Foundations first	The tokens, the state palette, the attention queue, the region model, the zoom tiers, the canvas↔world mapping and the shortcut registry are built once in Phase 1 and frozen at <code>rd-foundations</code> . A screen lane that needs a change files a request in <code>docs/redesign/requests.md</code> . The lead batches those requests into a small foundation fix between waves.
Contracts in Phase 0	<code>src/shared/redesign-contracts.ts</code> holds <i>types only</i> (AttentionItem, TaskRegion, ZoomTier, FloorPoint, ShortcutDef, StateColor). F2 and F3 can then start in parallel against the same interfaces, and later lanes code against stable shapes.

Mechanism	How it works
One owner per file per wave	<code>docs/redesign/ownership.json</code> maps globs to lanes. The <code>guard-paths.sh</code> PreToolUse hook blocks Edit/Write outside the lane's set and prints the owner.
Pre-cut seams	Phase 0 inserts named empty sections wherever several lanes must add lines: CSS blocks in <code>styles.css</code> (<code>/* — rd:L-B — */ ... /* — /rd:L-B — */</code>), one scenes file per lane that <code>shot.cjs</code> imports, one verify script per lane already registered in <code>package.json</code> , and channel slots in <code>ipc-contract.ts</code> . Lanes write only inside their own section, so git never sees two branches touching the same hunk.
Hotspot rule	<code>Canvas.tsx</code> (hook order matters: see <code>canvas/CLAUDE.md</code>) and <code>WorldView.tsx</code> each have exactly one lane owner per wave (L-B in P2a, L-C in P2b, W2 in P3b, W4 in P3c, W5 in P3d). Other lanes expose a component or hook, and the owner (or the lead) adds the one mounting line.

2.6 Verification gates

Gate	When	What runs	Who
G1	Every time a lane session stops	<code>npm run typecheck</code> (Stop hook) · <code>npm run affected</code> · the lane's <code>verify:rd-<lane></code> plain-node checks	Hooks + verify-runner
G2	Before asking to merge	Rebase · full <code>npm run verify</code> behind the Electron lock · <code>npm run shot</code> for the lane's scenes → <code>.vs-reference.png</code> · <code>mockup-critic</code> verdict per scene · <code>rules-reviewer</code> (and <code>world-guard</code> for P3) · the lane's manual checklist	Lane + subagents, then you look
G3	After each merge	Full <code>npm run verify</code> on <code>redesign/main</code> · <code>npm run build</code> · tag	Lead
G4	End of P2 and P3	Manual QA (§9.4) on a real machine with real claude/codex CLIs and <code>tmux</code> · <code>light-theme</code> pass · <code>reduced-motion</code> pass	You
G5	Release	<code>verify:visual</code> against new goldens (each with a critic sentence) · <code>verify:packaged</code> · signed package smoke test	Lead + you

VISUAL REFERENCE SCREENSHOTS VS. MOCKUPS

The harness already supports this. A scene in `scripts/shot.cjs` may declare `reference: [...]` repo paths (pinned by `verify:meta critic.reference.1`), and the shot run writes `<scene>.vs-reference.png`: the reference on top, the last golden beside the fresh capture beneath. Phase 0 extends `reference` to the 14 new `rd-*` scenes pointing at `docs/redesign/mockups/NN-*.png`. The mockups are 1440×900 and the shot window is 1440×865, so the critic judges *reading*, not pixel equality. Never use pixel diffs against the mockups. Pixel budgets apply only to the goldens.

Prerequisites and machine setup

3.1 Tools

Need	Why / check
macOS (Apple silicon), Xcode Command Line Tools	node-pty compiles natively against Electron's ABI. The app, the CI and the shot harness are all macOS-only. <code>xcode-select -p</code>
Node 20 (<code>engines: >=20</code>), npm	Matches CI (<code>actions/setup-node 20</code>). Use <code>nvm use 20</code> or <code>fnm</code> .
tmux (<code>brew install tmux</code>)	The session backend and the tmux suites. Without it, several suites skip loudly. You also host the lane sessions in tmux.
gh CLI, git ≥ 2.40	Worktrees and PRs into <code>redesign/main</code> .
Claude Code, latest (<code>claude update</code> ; check <code>claude --version</code>)	This plan uses subagent frontmatter fields such as <code>isolation</code> , <code>omitClaudeMd</code> and <code>effort</code> . Some of those need recent versions (<code>omitClaudeMd</code> needs v21.271+). Sign in with the Max 20x account, and use <code>/model</code> to put lanes on the strongest model.
Real agent CLIs on PATH: <code>claude</code> , <code>codex</code> (optional: <code>gemini</code>)	Needed for the onboarding PATH discovery (O2) and the manual QA on live states.
Disk: about 3–4 GB per worktree	Each worktree has its own node_modules with Electron and the native rebuild. Six worktrees at peak.

3.2 Baseline (do this before Claude touches anything)

TERMINAL · BASELINE

```
git clone git@github.com:alexnieves-cs/terminal-canvas.git ~/code/terminal-canvas
cd ~/code/terminal-canvas
git tag pre-redesign && git push origin pre-redesign
npm ci # postinstall: electron-rebuild -f -w node-pty
node node_modules/electron/install.js # the "Electron uninstall" gotcha; a no-op if present
unset ELECTRON_RUN_AS_NODE # VS Code's terminal exports it; dev/start unset it, scripts may not
npm run typecheck
time npm run verify 2>&1 | tee ~/rd-baseline-verify.log # record the duration and any reds
npm run shot # note the printed shot directory; keep it as the "before"
set
```

If anything is red on the baseline, record it in the ledger as *pre-existing*. A lane must never be blamed for, or try to "fix", a red it didn't cause.

3.3 Put the spec inside the repo

Copy the spec into tracked paths. Lanes and the critic can then read it, and `shot.cjs` `reference` fields can point at repo paths (required by `critic.reference.1`).

TERMINAL · SPEC INTO THE REPO (ON REDESIGN/MAIN)

```
git switch -c redesign/main pre-redesign
mkdir -p docs/redesign/mockups docs/redesign/lanes
cp /path/to/terminal-canvas-mockups/CONCEPT.md docs/redesign/CONCEPT.md
cp /path/to/terminal-canvas-mockups/[01][0-9]-*.png docs/redesign/mockups/
cp /path/to/terminal-canvas-implementation-guide.pdf docs/redesign/GUIDE.pdf
git add docs/redesign && git commit -m "docs(m435): redesign spec, mockups 01-14 and the
implementation guide"
```

SAMPLE COPY IS NOT PRODUCT COPY

The mockups use demo data ("Steward", "ledger-export", SW-412, \$3.12). That data belongs in the **shot fixtures** only. product-rules.md already lists "sample copy" as a reference feature the critic must not ask you to copy.

The Claude Code kit (created in Phase 0)

Everything below is committed to `redesign/main` in Phase 0, so every worktree inherits it. Claude Code features used: project memory (`CLAUDE.md`, nested), project subagents (`.claude/agents/`), custom slash commands (`.claude/commands/`; Claude Code now treats these as skills, and `.claude/skills/<name>/SKILL.md` is the equivalent newer layout), hooks in `.claude/settings.json`, permission rules, plan mode, and headless `claude -p`.

4.1 CLAUDE.md: the redesign section

Add one row to the root CLAUDE.md index table and one short section. Keep the root file an index, as its own header says. The detail goes in `docs/redesign/RUN.md`. After editing, run `npm run verify:meta`, because it pins parts of CLAUDE.md (ledger links, `claude-md.1`).

CLAUDE.MD · ADDED SECTION

The redesign run (M435-M456)

Spec: docs/redesign/CONCEPT.md + docs/redesign/mockups/01-14. Run book: docs/redesign/RUN.md.
Decisions (binding): docs/redesign/DECISIONS.md. Ledger: docs/build-log/m435-m456-ledger.md.

- You are in ONE lane. Your lane id is `$RD_LANE`; your brief is `docs/redesign/lanes/$RD_LANE.md`; the files you may edit are `docs/redesign/ownership.json[$RD_LANE]`. A hook enforces this.
- Foundations are frozen at tag `rd-foundations`: `src/shared/state-palette.ts`, `attention-queue.ts`, `world-space.ts`, `shortcuts.ts`, `redesign-contracts.ts`, `canvas/zoom-tier.ts`, `canvas/task-regions.ts`. Need a change? Append to `docs/redesign/requests.md` and stop; do not edit them.
- State colour comes ONLY from the Tone in `panels/panel-state.ts` -> `[data-tone]` / `state-palette.ts`. Never write a state hex or a state word anywhere else (`verify:rail state.2, rd-tone.*`).
- Write inside your pre-cut seams only: `styles.css` `"/* — rd:<lane> — */` blocks, `scripts/shot-scenes/rd-<lane>.cjs`, `scripts/verify-rd-<lane>.cjs`.
- Electron-tier verification ALWAYS through `scripts/redesign/with-electron-lock.sh`.
- Never run `UPDATE_GOLDENS=1`. Goldens are written by the lead after a person has looked.
- Before editing a module: `npm run lb -- <module>` (or ask the `lb-scout` subagent).

4.2 Subagents (`.claude/agents/`)

Only `name` and `description` are required. Keep descriptions short (they cost context in every session) and put the detail in the body. Tool names: `Read`, `Grep`, `Glob`, `Bash`, `Edit`, `Write`. The Task tool is now called `Agent`, and `Task(...)` still works as an alias.

`.CLAUDE/AGENTS/LB-SCOUT.MD`

```
---
name: lb-scout
description: Read-only. Before a module is changed, report its load-bearing rules, checks that pin it,
and seams. Use proactively before editing any existing file.
tools: Read, Grep, Glob, Bash
model: inherit
---
You map constraints; you never edit. For each module you are given:
1. Run `npm run lb -- <module basename>` and read every entry it prints (whole entries, file:line).
2. Read the nearest CLAUDE.md (root, src/main, src/renderer, src/renderer/canvas) and the
   module's rows in docs/architecture-map.md and docs/product-rules.md.
3. rg the scripts/ directory for checks that read this file as TEXT or select its DOM classes.
Return at most 25 lines: MUST-KEEP rules (with the check id that pins each), DOM aliases that
must not be renamed, hook-order or import-direction rules, and the safest seam for the change.
```

`.CLAUDE/AGENTS/CHECK-AUTHOR.MD`

```
---
name: check-author
description: Writes verify checks in this repo's idiom (plain-node first, scoped ids). Use when a step
needs new acceptance checks.
tools: Read, Grep, Glob, Bash, Edit, Write
model: inherit
---
You add checks to scripts/verify-rd-<lane>.cjs (already registered in package.json) or append to an
existing suite only when the brief says so. Rules: ids are SCOPED ('ok('rd-tiers.3 ...')'), never a
global integer; pure logic is checked in plain node against the pure module (viewport/lod style);
renderer behaviour goes in the Electron tier only when a pure check cannot observe it; each check
must be watched RED once (break the code, see it fail, restore) and you say how in a comment.
Never restate a count in prose. Report the ids you added and the red-then-green evidence.
```

`.CLAUDE/AGENTS/VERIFY-RUNNER.MD`

```
---
name: verify-runner
description: Runs a named gate (G1 or G2) and returns only failures with their check ids and likely
cause. Use for any verify, affected, typecheck or shot run.
tools: Bash, Read, Grep, Glob
model: inherit
---
Gates:
- G1: `npm run typecheck` then `npm run affected` then `npm run verify:rd-$RD_LANE`.
- G2: `git fetch && git rebase origin/redesign/main` (stop and report on conflict), then
    `scripts/redesign/with-electron-lock.sh npm run verify`, then
    `scripts/redesign/with-electron-lock.sh env TC_SHOT_ONLY=<lane scenes> npm run shot`.
Return: PASS/FAIL per step, each failing check id + its first error lines, the slowest-suite table
tail, and the shot directory path. Never edit code. Never set UPDATE_GOLDENS. If a red is listed as
pre-existing in docs/build-log/m435-m456-ledger.md, say so instead of treating it as new.
```


`.CLAUDE/AGENTS/MOCKUP-CRITIC.MD`

```
---
name: mockup-critic
description: Fresh-context visual critic. Judges a <scene>.vs-reference.png composite against its
mockup. Use after every shot run of a redesign scene.
tools: Read, Glob
model: inherit
omitClaudeMd: true
---
```

You have no repo context on purpose. You are given composite images (reference on top; last golden and fresh capture below) and docs/redesign/CONCEPT.md. For each scene answer ONE question: does the capture read as the reference? List every divergence in this order: silhouette, material, glow, connectors, labels, composition, chrome, then STATE COLOUR (cyan working, amber needs-you and the only glow, green done, red failed, slate idle; World uses the same). Ignore sample copy (Steward, ledger-export, SW-412, dollar amounts), 35px height difference, and anything CONCEPT.md says is deliberately not copied. Output: VERDICT reads-as / close / does-not-read, then a numbered list, most important first, each with where on screen. Quote nothing you cannot see.

`.CLAUDE/AGENTS/RULES-REVIEWER.MD`

```
---
name: rules-reviewer
description: Read-only review of a lane diff against product rules and house rules. Use before G2.
tools: Read, Grep, Glob, Bash
model: inherit
---
```

Review 'git diff origin/redesign/main...HEAD'. Check, citing file:line: face rule (mono only on code/paths/cells; no mono ancestor), rest rule (one state at rest; verbs opacity 0->1 in --dur-1, never display:none), metrics rule as amended by DECISIONS.md D3, words-not-codes empty states via EmptyState/empty-states.ts, state words/tones only from panel-state.ts, no state hex outside state-palette.ts/styles.css token blocks, new CSS tokens in BOTH theme blocks, DOM aliases (.panel__*, *-node__*) not renamed, .pf__body never transformed, chromeless chrome absolutely positioned, library doors (renderer/CLAUDE.md), scoped verify ids, files outside ownership.json. Output: BLOCKERS, SHOULD-FIX, NITS. No praise.

`.CLAUDE/AGENTS/WORLD-GUARD.MD`

```
---
name: world-guard
description: Read-only guard for 3D World changes - bundle doors, lazy boundary, CSP, WebGL budget,
reduced motion, colour parity. Use on every Phase 3 diff.
tools: Read, Grep, Glob, Bash
model: inherit
---
```

Check: three/@react-three/fiber/drei/postprocessing imported only by the files listed in src/renderer/CLAUDE.md's three rows (update that table in the same diff if a file is added); WorldStage.tsx keeps the ONE '/* @__PURE__ */ lazy()'; pure modules (world-scene, world-set, world-roster, world-palette, world-transition, world-toggle, world-facts, world-space) import no three; no drei <Environment>, useGLTF with decoders off; colours come from src/shared/state-palette.ts (no literal state hex in world/*); prefers-reduced-motion handled (flights collapse, 120 ms cross-fade); WebGL context count (xterm LIVE_BUDGET 8 + world) stays under ~16 and context loss falls back to WorldFlat. Run 'npm run verify:world' and 'npm run build', then check the built entry does not statically import three (orch-zoom.3). Output BLOCKERS / SHOULD-FIX.

4.3 Slash commands (`.claude/commands/`)

`$ARGUMENTS` takes everything typed after the command name. Commands can't switch the permission mode, so start lane sessions in plan mode with `claude --permission-mode plan`, or press Shift+Tab to cycle modes.

`.CLAUDE/COMMANDS/RD-LANE.MD`

```
---
description: Load this lane's brief and produce a plan (use in plan mode)
---
You are lane $RD_LANE of the Terminal Canvas redesign. Read, in order: CLAUDE.md (redesign section),
docs/redesign/DECISIONS.md, docs/redesign/lanes/$RD_LANE.md, docs/redesign/ownership.json (your
entry), docs/redesign/CONCEPT.md, and every mockup PNG your brief names (look at them).
Then use the lb-scout subagent on every existing file your brief lists, in parallel.
Produce a plan: milestones (Mxxx) in order; per milestone the files, the pure model first, the
checks (ids) that will prove each acceptance criterion, and the shot scenes. Flag any file you need
that you do not own, and any foundation change (goes to docs/redesign/requests.md). Extra focus:
$ARGUMENTS
```

`.CLAUDE/COMMANDS/RD-GATE.MD`

```
---
description: Run gate G1 or G2 via the verify-runner subagent
argument-hint: [G1|G2]
---
Use the verify-runner subagent to run gate $ARGUMENTS for lane $RD_LANE. If G2 passes its
automated part, run the mockup-critic subagent on each composite the shot run wrote for this
lane's rd-* scenes and the rules-reviewer on the diff (plus world-guard if the lane starts with W).
Summarise: gate result, critic verdicts with top 3 divergences per scene, reviewer blockers.
Do not fix anything in this command; propose the fix list.
```

`.CLAUDE/COMMANDS/RD-CRITIC.MD`

```
---
description: Fresh-context critique of one or more shot composites against the mockups
argument-hint: <shot-dir or composite paths>
---
Run the mockup-critic subagent once per composite in $ARGUMENTS, in parallel. Return each verdict
verbatim, then append them verbatim to docs/build-log/m435-m456-ledger.md under the lane's
current milestone heading, with any disagreement written beside the divergence (product-rules.md
"The critic and the reference").
```

`.CLAUDE/COMMANDS/RD-MERGE.MD`

```
---
description: LEAD ONLY - merge a lane into redesign/main and run G3
argument-hint: <lane-branch>
---
Refuse unless the current branch is redesign/main and RD_LANE is "lead". Steps: fetch; confirm
the lane's PR shows G2 results and critic sentences in the ledger; 'git merge --no-ff $ARGUMENTS';
on conflict stop and list the hunks (do not resolve foundation files by guessing);
'scripts/redesign/with-electron-lock.sh npm run verify'; 'npm run build'; update the ledger's merge
table; print the rebase command for every other live lane in this wave.
```

4.4 Hooks and permissions (`.claude/settings.json`)

`.CLAUDE/SETTINGS.JSON`

```
{
  "permissions": {
    "allow": ["Bash(npm run *)", "Bash(npx tsc *)", "Bash(git status*)", "Bash(git diff*)",
      "Bash(git log*)", "Bash(git add *)", "Bash(git commit *)", "Bash(rg *)",
      "Bash(scripts/redesign/*)"],
    "ask": ["Bash(git push *)", "Bash(git rebase *)", "Bash(git merge *)"],
    "deny": ["Bash(git push --force*)", "Bash(git push -f*)", "Bash(*UPDATE_GOLDENS=1*)",
      "Bash(git reset --hard*)"]
  },
  "worktree": { "baseRef": "head" },
  "hooks": {
    "SessionStart": [{ "hooks": [{ "type": "command",
      "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/session-brief.sh" }] }],
    "PreToolUse": [
      { "matcher": "Edit|Write|MultiEdit", "hooks": [{ "type": "command",
        "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/guard-paths.sh" }] }
    ],
    "PostToolUse": [
      { "matcher": "Edit|Write|MultiEdit", "hooks": [{ "type": "command",
        "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/quick-check.sh" }] }
    ],
    "Stop": [{ "hooks": [{ "type": "command",
      "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/stop-typecheck.sh" }] }],
    "Notification": [{ "hooks": [{ "type": "command",
      "command": "osascript -e 'display notification \"Lane needs you\" with title \"Claude Code\"'"
    }] }]
  }
}
```

`.CLAUDE/HOOKS/GUARD-PATHS.SH (CHMOD +X) - BLOCKS EDITS OUTSIDE THE LANE'S OWNERSHIP`

```
#!/bin/bash
# PreToolUse: stdin is JSON with tool_input.file_path. Exit 2 = block, stderr goes back to Claude.
INPUT=$(cat)
node -e '
  const i = JSON.parse(process.argv[1]); const lane = process.env.RD_LANE || "";
  const f = (i.tool_input && (i.tool_input.file_path || i.tool_input.path)) || "";
  const root = process.env.CLAUDE_PROJECT_DIR; const rel = require("path").relative(root, f);
  if (!lane || lane === "lead") process.exit(0);
  if (rel.startsWith("verify/visual/goldens/")) { console.error("Goldens are lead-only.");
process.exit(2) }
  const own = require(root + "/docs/redesign/ownership.json");
  const mm = (g, p) => new RegExp("^" + g.replace(/[\.\^\$\{\}\|\[\]\*\]/g, "\\$&")
    .replace(/\/\*\*/g, "\\u0000").replace(/\/\*/g, "[^/]*").replace(/\\u0000/g, ".") + "$").test(p);
  const mine = (own.lanes[lane] || []).concat(own.shared_writable || []);
  if (mine.some(g => mm(g, rel))) process.exit(0);
  const owner = Object.entries(own.lanes).find(([, gs]) => gs.some(g => mm(g, rel)));
  console.error(`${rel} is not owned by ${lane}` + (owner ? ` (owner: ${owner[0]})` : "") +
    ". Expose a hook/component in your own files, or append a request to docs/redesign/requests.md.");
  process.exit(2);
  ' "$INPUT"
```

```
.CLAUDE/HOOKS/STOP-TYPECHECK.SH (CHMOD +X)
```

```
#!/bin/bash
# Stop: keep Claude working until the tree typechecks. stop_hook_active prevents an infinite loop.
INPUT=$(cat)
if echo "$INPUT" | grep -q '"stop_hook_active": *true'; then exit 0; fi
cd "$CLAUDE_PROJECT_DIR" || exit 0
OUT=$(npm run -s typecheck 2>&1) || { echo "Typecheck failed - fix before stopping:" >&2;
  echo "$OUT" | tail -40 >&2; exit 2; }
exit 0
```

```
.CLAUDE/HOOKS/QUICK-CHECK.SH AND SESSION-BRIEF.SH (SKETCH)
```

```
# quick-check.sh: cheap, targeted plain-node checks after an edit (never the Electron tier)
case "$FILE" in
  *styles.css)      npm run -s verify:styles ;;
  *panel-state.ts|*rail*) npm run -s verify:rail ;;
  src/renderer/world/*) npm run -s verify:world ;;
  src/renderer/palette/*) npm run -s verify:palette ;;
esac # print failures to stderr and exit 2 so Claude sees them; exit 0 otherwise

# session-brief.sh: stdout is added to the session context at start
echo "Lane: $RD_LANE Branch: $(git branch --show-current) Base: $(git describe --tags --abbrev=0)"
echo "Owned globs:"; node -e
"console.log((require('./docs/redesign/ownership.json').lanes[process.env.RD_LANE]||[]).join('\n'))"
echo "Open requests:"; tail -n 15 docs/redesign/requests.md 2>/dev/null
```

4.5 Run scripts (`scripts/redesign/`)

```
SCRIPTS/REDESIGN/LANE.SH - CREATE A LANE WORKTREE AND OPEN IT IN TMUX, IN PLAN MODE
```

```
#!/bin/bash
# usage: scripts/redesign/lane.sh l-b-workspace L-B [base-tag]
set -euo pipefail
NAME=$1; LANE=$2; BASE=${3:-$(git describe --tags --abbrev=0 redesign/main)}
DIR=../tc-rd-$NAME
git worktree add "$DIR" -b "rd/$NAME" "$BASE"
( cd "$DIR" && npm ci && node node_modules/electron/install.js )
tmux has-session -t rd 2>/dev/null || tmux new-session -d -s rd -n lead
tmux new-window -t rd -n "$LANE" -c "$DIR" \
  "RD_LANE=$LANE claude --permission-mode plan; exec \$SHELL"
echo "Attach: tmux attach -t rd (window $LANE)"
```

```
#!/bin/bash
# The harness uses FIXED fixture directory names and one verify tmux socket, and
# TC_VERIFY_ELECTRON_JOBS>1 is measured unreliable. Two worktrees running Electron suites at
# once corrupt each other's results. macOS has no flock(1): use an atomic mkdir lock.
LOCK=/tmp/tc-rd-electron.lock
until mkdir "$LOCK" 2>/dev/null; do
  PID=$(cat "$LOCK/pid" 2>/dev/null); if [ -n "$PID" ] && ! kill -0 "$PID" 2>/dev/null; then rm -rf
"$LOCK"; continue; fi
  echo "waiting for Electron lock (held by pid $PID: $(cat $LOCK/who 2>/dev/null))"; sleep 15
done
echo $$ > "$LOCK/pid"; echo "${RD_LANE:-?} $(pwd)" > "$LOCK/who"
trap 'rm -rf "$LOCK"' EXIT
unset ELECTRON_RUN_AS_NODE
"$@"
```

4.6 Headless mode, where it pays off

- **Batch critic.** After a shot run, critique every composite without tying up a session: `for f in "$SHOT"/rd-*.vs-reference.png; do claude -p "Use the mockup-critic subagent on $f. Return only its output." --output-format json --allowedTools "Read,Glob,Agent" > "${f%.png}.critic.json"; done`
- **Morning digest (lead).** `claude -p "For each worktree under ../tc-rd-*, summarise git log since yesterday, last G1/G2 result in the ledger, and open requests.md items" --output-format text`
- **Plan-only dry runs.** `claude -p --permission-mode plan "/rd-lane"` in a fresh worktree, to see whether a lane brief is clear before you spend a live session on it.

OPTIONAL CLAUDE CODE FEATURES, USED ON PURPOSE

Claude Code also has **agent teams**, **agent view** for dispatching background sessions, and the bundled **/batch** skill, which splits a change into many worktree-isolated subagents that each open a PR. This plan uses explicit lanes instead, because file ownership and merge order are the main risk in this repo. `/batch` is a good fit for mechanical sweeps inside one lane, for example "replace every literal state hex in `src/renderer/world/` with `state-palette.ts` reads".

Run kit, seams and contracts

SEQ

MILESTONE M435	WHO Lead session (RD_LANE=lead)	SUBAGENTS lb-scout x4 in parallel	EXIT G3 green · tag rd-p0
-------------------	---------------------------------------	--------------------------------------	------------------------------

Goal. Make parallel work safe before any feature code exists. When Phase 0 ends, every later lane can start from `rd-p0`, read its own brief, and write only inside its own files and seams.

Files created or touched

Path	Purpose
<code>docs/redesign/{RUN,DECISIONS,requests}.md</code> , <code>ownership.json</code> , <code>lanes/*.md</code>	The run book, the D1–D8 answers, the foundation-change queue, file ownership per lane and wave, and one brief per lane transcribed from Sections 6–8 of this guide.
<code>.claude/agents/*</code> , <code>.claude/commands/*</code> , <code>.claude/settings.json</code> , <code>.claude/hooks/*</code>	The kit from Section 4.
<code>scripts/redesign/{lane,with-electron-lock}.sh</code>	Lane creation and the Electron lock.
<code>src/shared/redesign-contracts.ts</code>	Types only: <code>StateTone</code> re-export, <code>AttentionItem</code> , <code>TaskRegion</code> , <code>ZoomTier</code> , <code>FloorPoint</code> / <code>CameraPose</code> , <code>ShortcutDef</code> . No logic.
<code>src/renderer/styles.css</code> (markers only)	One empty <code>/* — rd:<lane> — */ ... /* — /rd:<lane> — */</code> block per lane (F1..W6), with an unchanged line between blocks.
<code>scripts/shot.cjs</code> + <code>scripts/shot-scenes/rd-*.cjs</code>	Each lane's scenes live in its own file, which <code>shot.cjs</code> concatenates. Add a <code>TC_SHOT_ONLY=a,b</code> filter if none exists. Let <code>rd-*</code> scenes declare <code>reference</code> (extend <code>critic.reference.1</code> to cover them).
<code>scripts/verify-rd-*.cjs</code> + <code>package.json</code> <code>verify:rd-*</code>	One stub suite per lane with <code>ok('rd-<lane>.0 seam present')</code> , all registered now, so no lane ever edits <code>package.json</code> scripts. Update <code>docs/verify-suite-table.md</code> if <code>verify:meta</code> pins it.
<code>docs/build-log/m435-m456-ledger.md</code> + root CLAUDE.md link	The ledger starts with the baseline reds and the decisions. The link keeps <code>verify:meta ledger.1</code> green.

```
{
  "shared_writable": ["docs/build-log/m435-m456-ledger.md", "docs/redesign/requests.md"],
  "lanes": {
    "F1": ["src/rendererer/styles.css", "src/rendererer/panels/panel-state.ts", "src/shared/state-
palette.ts",
          "src/rendererer/primitives/StatusDot.tsx", "src/rendererer/shell/chart-tokens.ts",
          "scripts/verify-rd-f1.cjs", "scripts/shot-scenes/rd-f1.cjs", "scripts/verify-styles.cjs"],
    "F2": ["src/shared/attention-queue.ts", "src/shared/world-space.ts", "src/rendererer/canvas/task-
regions.ts",
          "src/rendererer/canvas/zoom-tier.ts", "src/rendererer/session/useAttentionQueue.ts",
          "scripts/fixtures/rd-steward/**", "scripts/verify-rd-f2.cjs"],
    "L-B": ["src/rendererer/canvas/Canvas.tsx", "src/rendererer/panels/**",
"src/rendererer/canvas/TaskClusterLayer.tsx", "..."],
    "W2": ["src/rendererer/world/WorldStructure.tsx", "src/rendererer/world/WorldOffice.tsx", "..."]
  },
  "css_sections": { "styles.css": "lanes write only between their rd:<lane> markers (F1 also owns the
token blocks)" }
}
```

Acceptance criteria

- ☐ `npm run verify` is green apart from the reds recorded as pre-existing. Every `verify:rd-*` stub runs and passes.
- ☐ `npm run shot` still produces every existing scene. `TC_SHOT_ONLY=launcher npm run shot` produces only that scene.
- ☐ From a test worktree with `RD_LANE=F1`, an Edit to `src/rendererer/world/WorldView.tsx` is blocked by the hook, and the message names the owner.
- ☐ Two worktrees running `with-electron-lock.sh npm run verify:canvas` at once run one after the other.
- ☐ CLAUDE.md links the ledger. `verify:meta` is green.

You are the LEAD of the Terminal Canvas redesign run (M435-M456). You do not build features; you build the conditions for parallel lanes to build them safely.

Read first: CLAUDE.md, src/rendererer/CLAUDE.md, src/rendererer/canvas/CLAUDE.md, src/main/CLAUDE.md, docs/product-rules.md, docs/verify-suites.md (its five silent rules), docs/redesign/CONCEPT.md, and docs/redesign/GUIDE.pdf sections 2, 4 and 5 (the implementation guide; read it with your Read tool). In parallel, use four lb-scout subagents on: scripts/shot.cjs, scripts/verify-all.cjs + scripts/verify-meta.cjs, src/rendererer/styles.css (theme blocks + [data-tone]), package.json.

Then plan Phase 0 exactly as GUIDE.pdf section 5 lists it:

1. docs/redesign/DECISIONS.md from my answers below; RUN.md; requests.md (empty, with a template).
2. docs/redesign/ownership.json for EVERY lane (F1 F2 F3 L-A L-B L-C L-D L-E L-F W0-W6), by wave, taken from the "Files likely touched" tables in sections 6-8 of GUIDE.pdf. No file owned twice in one wave; Canvas.tsx and WorldView.tsx follow the hotspot rule in section 2.5.
3. docs/redesign/lanes/<LANE>.md: transcribe each lane's goal, files, acceptance criteria, mockups and prompts from GUIDE.pdf verbatim, so a lane never has to open the PDF.
4. The .claude/ kit and scripts/redesign/ scripts from section 4 (make hooks executable; test guard-paths.sh with a fake RD_LANE; test the lock with two shells).
5. The pre-cut seams: styles.css markers; scripts/shot-scenes/rd-<lane>.cjs + TC_SHOT_ONLY + reference support for rd-* scenes; scripts/verify-rd-<lane>.cjs stubs registered as verify:rd-<lane>; channel slot comments in src/shared/ipc-contract.ts.
6. src/shared/redesign-contracts.ts – types only, as listed in section 5.
7. docs/build-log/m435-m456-ledger.md with the baseline (~/.rd-baseline-verify.log) and the root CLAUDE.md row + redesign section (section 4.1). Run verify:meta.

Gate: scripts/redesign/with-electron-lock.sh npm run verify, then commit "chore(m435): ..." in small commits and tag rd-p0. Stop and show me the plan before writing anything.

My decisions (D1-D8): <paste your answers, or "accept the guide's defaults">

Foundations

PAR x3

Three lanes run in parallel against the Phase 0 contracts, and they merge in the order **F1 → F2 → F3**. After the merge, the files they created are frozen under tag `rd-foundations`. Every screen in 01–14 depends on these, so nothing else starts until the tag exists.

PHASE 1 · LAUNCH THE THREE FOUNDATION LANES

lead, in a plain terminal

```
scripts/redesign/lane.sh f1-tokens F1 rd-p0
scripts/redesign/lane.sh f2-model F2 rd-p0
scripts/redesign/lane.sh f3-frame F3 rd-p0
tmux attach -t rd # one window per lane; each opened claude in plan mode
```

EVERY LANE · THE STANDARD KICKOFF (PASTE IN EACH LANE WINDOW, THEN ADD THE LANE'S SPECIFIC PROMPT)

plan mode

```
/rd-lane
When I approve the plan: implement it milestone by milestone. For each milestone:
(1) write the pure model and its checks first with the check-author subagent, and watch each check
fail before the code exists; (2) build the UI on top; (3) run /rd-gate G1 and fix until green;
(4) commit as feat(mNNN): ... with the ledger entry. Use subagents in parallel for independent reads.
Keep every edit inside docs/redesign/ownership.json for $RD_LANE; if the guard blocks you, do not
work around it – expose a hook/component in your own files or append to requests.md.
When all milestones are done, run /rd-gate G2 and stop. Do not merge.
```

F1 · Design tokens and the one state colour system

MILESTONE	LANE · BRANCH	DEPENDS ON	MOCKUPS
M436	F1 · rd/f1-tokens	rd-p0 · D1, D7	02 (preview), 04, 07, 11

Goal. Every surface reads state colour from one closed set. A Tone in `panel-state.ts` maps to a `--state-*` token in `styles.css` and to a hex in `src/shared/state-palette.ts` for WebGL and canvas-2D readers. The Obsidian material is re-valued to the concept table.

Tone (panel-state.ts)	Word	Token	Dark value	Rule
<code>working</code> (+ <code>starting</code> , breathing)	working	<code>--state-working</code>	 #5BE1E6	Also the brand accent and focus (D1 decides the selection treatment).
<code>needs-you</code>	needs you	<code>--state-needs</code>	 #F5B544	The only tone that glows (edge + soft glow; in the World, the only vertical beacon).
<code>done</code> (D7: watcher passed, work done, auto done)	exited 0 / passed / done	<code>--state-done</code>	 #5EDC9A	Green means finished OK. It never means "working" again (the M415 inversion ends).
<code>exited</code> (non-zero, signal, error)	exited N / failed	<code>--state-failed</code>	 #FF6B6B	Paired with one fix verb (O9).

Tone (panel-state.ts)	Word	Token	Dark value	Rule
idle, asleep, none, kind	idle / asleep / not started / kind	--state-idle	#8C94A5	Slate. Asleep keeps its dashed left edge.
(not a tone) external / Codex · smart guides	—	--external · --guide	#A78BFA · #FF5FA2	Guides appear only while arranging, never at rest.

Material re-valuation (dark theme): ink #08090D, canvas #0B0D12 with a 24px dot grid #1B202A, surfaces #11141B / #161A23 / #1C2130, hairlines rgba(255,255,255,.07/.12), text #E7EAF0 / #8C94A5 / #5B6375. Radii: panels 12, controls 8, regions 20, pills full. Add --dur-flight: 220ms with ease-out. Light-theme values are re-derived against contrast check 11, not copied.

FILES LIKELY TOUCHED

src/renderer/styles.css (both theme blocks, the [data-tone] block near the "ONE status dot" rule, .status-dot), src/renderer/panels/panel-state.ts (tones for watcher, work and auto per D7; MINIMAP_LEGEND gains done), src/shared/state-palette.ts (new), src/renderer/primitives/StatusDot.tsx, src/renderer/shell/chart-tokens.ts, scripts/verify-rd-f1.cjs, scripts/verify-styles.cjs (only if a check pins old values).

ACCEPTANCE CRITERIA

- The [data-tone] block reads only --state-* tokens. Each new name is declared in both theme blocks (verify:styles theme.1).
- rd-tone.parity.1: the hexes in the styles.css token blocks equal state-palette.ts for both themes.
- rd-tone.glow.1: no glow, box-shadow halo or filter rule applies to any tone except needs-you (selection excluded per D1).
- rd-tone.literal.1: no state hex appears in src/renderer/** outside styles.css token blocks and state-palette.ts. A world exemption list exists until W0 removes it.
- D7 is applied. verify:rail (state.2) stays green, and words are unchanged.
- The O2 preview card, the O4 frame edges and the O7 state pills read the same five colours in the rd-f1-tones shot scene (a one-of-each-tone board). The critic verdict is "reads-as" against O4 for state colour.

F1 · LANE-SPECIFIC PROMPT

after the standard kickoff

Lane F1 (M436): the design tokens and the one state colour system, per GUIDE.pdf S6 F1 and DECISIONS.md D1/D7. Start with src/shared/state-palette.ts (pure, no DOM: both themes' hexes per Tone + external + guide) and the four rd-tone.* checks. Then re-value the dark theme to CONCEPT.md's table and add the --state-* tokens to BOTH theme blocks; derive light values against check 11 and record every re-valuation with its reason in the ledger. Point the [data-tone] block at the new tokens. Apply D7 inside panel-state.ts only (watchState passed, WORK_TONES done, autoTone done -> 'done'; words unchanged). Apply D1's selection ring. Add a shot scene rd-f1-tones (one panel per tone, plus selection on a working panel) with reference docs/redesign/mockups/04-main-workspace.png. Do NOT touch world/*: W0 consumes state-palette.ts later. List every existing golden whose pixels will change, so the lead can plan Phase 4.

F2 · The shared model: attention queue, regions, zoom tiers, canvas↔world space

MILESTONE M437	LANE · BRANCH F2 · rd/f2-model	DEPENDS ON rd-p0 contracts	MOCKUPS 04, 06, 07, 11, 13, 14
-------------------	-----------------------------------	-------------------------------	-----------------------------------

Goal. Every surface asks the same pure modules what needs you, what belongs to which task, which zoom tier you are in, and where a canvas point sits on the World floor. That is how "one attention story" and "one spatial truth" become true by construction.

Module (new)	Contract
<code>src/shared/attention-queue.ts</code>	<code>buildQueue(inputs) → AttentionItem[]</code> , ordered and pure. Kinds: <code>approval</code> , <code>question</code> , <code>shell-prompt</code> , <code>failed</code> , <code>recovery</code> . Each item has a sentence, a since time, a task id and a primary verb. <code>next(id)</code> / <code>prev(id)</code> for ⌘J / ⌘↑J. It reads the same facts <code>panelState</code> reads, so the words cannot disagree.
<code>src/renderers/session/useAttentionQueue.ts</code>	The one selector, subscribed to <code>agent-state-store</code> , <code>live-session-store</code> and the decision inbox. The pill, dock badge, Sessions tab count, edge pips, minimap dots and World all read it.
<code>src/renderers/canvas/task-regions.ts</code>	Builds on <code>task-clusters.ts</code> and <code>task-members.ts</code> . Region bounds = union of member rects + 24px padding, snapped to the 24px grid, 20px radius. Label = ticket, title, agent count, criteria done/total. It also provides hit-testing and "move region moves members" as one undo.
<code>src/renderers/canvas/zoom-tier.ts</code>	<code>tierFor(scale, prev) : Work ≥ 0.70, Plan 0.25–0.70, Map < 0.25</code> , with ±0.03 hysteresis. <code>TIER_TARGET</code> scales for ⌘1/2/3. <code>cardIdsForTier()</code> feeds <code>lod.ts</code> 's existing <code>cardIds</code> input, so no PTY lifecycle changes .
<code>src/shared/world-space.ts</code>	<code>canvasToFloor(p) = {x: p.x/100, z: p.y/100}</code> and its inverse. <code>viewportToCamera(viewport, size, tier)</code> / <code>cameraToViewport(...)</code> round-trip within 1 canvas px. Pitch per tier: Work close, Plan room view, Map near top-down.
<code>scripts/fixtures/rd-steward/</code>	The demo workspace from the mockups: five tasks (Ledger CSV export SW-412, Plaid webhook retry SW-398, Infra, Steward web · pricing, Steward mobile · onboarding crash SW-421) with agents in every state, plus a <code>world-sim.ts</code> feed script for the World scenes. Fixture only; never product copy.

ACCEPTANCE CRITERIA

- All modules are pure and checked in plain node (`verify:rd-f2`): queue order and stability, `next` / `prev` wraparound, region bounds and snap, tier hysteresis (no flapping at 0.70±0.02), and world-space round-trip error < 1px over 1,000 random points and viewports.
- `rd-attn.one.1` (source-text check): the pill, dock badge and Sessions count import `useAttentionQueue`. None of them computes its own count.
- Entering Plan or Map with a focused terminal first releases focus, using the same path as ⌘Esc. A check proves no keystroke can reach a card (the `lod.ts` invariant).

- The fixture loads in `npm run dev` with `TC_FIXTURE=rd-steward` (or the harness's existing fixture switch), and the shot scenes can use it.

F2 · LANE-SPECIFIC PROMPT

after the standard kickoff

Lane F2 (M437): the shared model, per GUIDE.pdf §6 F2, against the types in `src/shared/redesign-contracts.ts` (do not change them without a `requests.md` entry the lead accepts). Use `lb-scout` first on `canvas/attention.ts`, `task-clusters.ts`, `task-members.ts`, `lod.ts`, `tier-fade.ts`, `viewport.ts`, `shell/decision-inbox.ts`, `shell/task-queue.ts` and `session/agent-state-store.ts`, in parallel.

Build the five modules `pure-first`, with `verify:rd-f2` checks written by `check-author` and watched `red`. Zoom tiers must compose with `lod.ts` through its existing `cardIds` input (never change `LIVE_BUDGET`, `LIVE_MIN_SCALE`, dormancy precedence or the focused-panel rule); entering `Plan/Map` releases focus. Then build the `rd-steward` fixture from the mockups (read 04, 06, 07, 11 and 14 to get the tasks, agents and states right) and a world-sim feed for it. Do not wire any UI: F3 and the screen lanes consume these. Document each module's contract in its header the way `world-transition.ts` does.

F3 · The shell frame: nav, shortcut registry, focus lock, pill

MILESTONE M438	LANE · BRANCH F3 · rd/f3-frame	DEPENDS ON contracts (F2 merged before F3 merges)	MOCKUPS 04 (title bar, pill), 07 (tab badge), 08, 11 (pill)
--------------------------------------	--	---	---

Goal. The chrome every screen shares: `Canvas | Sessions | Review` in the title bar, one shortcut registry that every chord and `Settings > Keyboard` read, focus lock, and the attention pill's rest line.

FILES LIKELY TOUCHED

`src/renderer/shell/TopBar.tsx`, `useShellChrome.ts` (`CenterView` gains `sessions` and `review`; `Orchestrate` is placed per D2), `src/shared/shortcuts.ts` (new registry), `src/renderer/canvas/useKeyboardNav.ts` (reads the registry), `src/main/menu.ts` (accelerators from the registry; `⌘T` kept as an alias per D5), `src/renderer/canvas/CommandPill.tsx` + `command-pill.ts`, `src/renderer/shell/Dock.tsx` (badge), new `src/renderer/sessions/SessionsHost.tsx` and `review/ReviewHost.tsx` mount points (empty hosts that L-D and the Review owner fill).

ACCEPTANCE CRITERIA

- The title bar matches 04: the product mark and workspace crumb on the left; the centred segmented `Canvas | Sessions ⓘ | Review`, with the badge count from the attention queue; then "7 sessions" (D3), "Find or run anything ⌘K" and the avatars on the right.
- `shortcuts.ts` lists every chord in CONCEPT.md "Key interactions" with id, chord, scope, group (create, sessions, navigate, mouse), label and handler. `rd-keys.dup.1` fails on a duplicate chord within a scope. `rd-keys.menu.1`: menu accelerators come from the registry.
- Focus lock (`⌘⇧L`): while a panel is locked, `shouldIgnoreKeys()` is true for every canvas chord except `⌘⇧L`. The panel shows a lock mark. `⌘K` then reaches the terminal (D4).
- The pill's rest line follows priority order and never shows a zero: "2 agents need you · Codex wants to edit ledger.ts · Go ⌘J · + New ⌘N" (04). It acts only through the palette's executor (the existing M249 rule).
- The existing `verify:panels` checks on `.shell__center-toggle`, `.shell__merge` and the View menu stay green: restyle, don't rename.

Lane F3 (M438): the shell frame, per GUIDE.pdf §6 F3 and DECISIONS.md D2-D5. lb-scout first on TopBar.tsx, useShellChrome.ts, useKeyboardNav.ts, main/menu.ts, CommandPill.tsx, command-pill.ts, Dock.tsx and palette/usePalette.ts (the Cmd+K rule reads event.code).

- 1) src/shared/shortcuts.ts: every chord from CONCEPT.md "Key interactions" plus today's existing chords (`%N`, `%T` alias, `%F`, `%I`, `%Space`, `%F`, `%\`, `%\`), with the known external conflict list (`%K` = clear in claude/terminals). Checks rd-keys.*. Migrate useKeyboardNav and menu.ts to read it without changing behaviour (existing verify:panels nav checks must stay green).
- 2) Focus lock `%L` as specified, with a check proving canvas chords stand down while locked.
- 3) CenterView sessions/review + TopBar per mockup 04; keep every existing class hook.
- 4) Pill rest line from useAttentionQueue (F2's contract; stub the import against the contract until F2 merges, then rebase).

Shot scenes: rd-f3-titlebar (reference 04) and rd-f3-pill.

MERGE AND FREEZE

The lead merges F1, then F2, then F3 (F3 rebases onto F2 first), runs G3, and tags `rd-foundations`. In ownership.json the foundation files move to `"frozen"`. From here on, the guard blocks every lane from editing them. Changes go through `requests.md`, and the lead lands them between waves.

Canvas screens 01–09

This phase runs in two waves. **Wave 2a** runs four lanes in parallel: L-A (01–03), L-B (04→05), L-D (07) and L-E (08). **Wave 2b** starts when L-B has merged and runs L-C (06) and L-F (09), which render on top of L-B's frame and regions. The steps below are in the order you run them, so screen 06 comes after 07 and 08.

WAVE 2A · LAUNCH

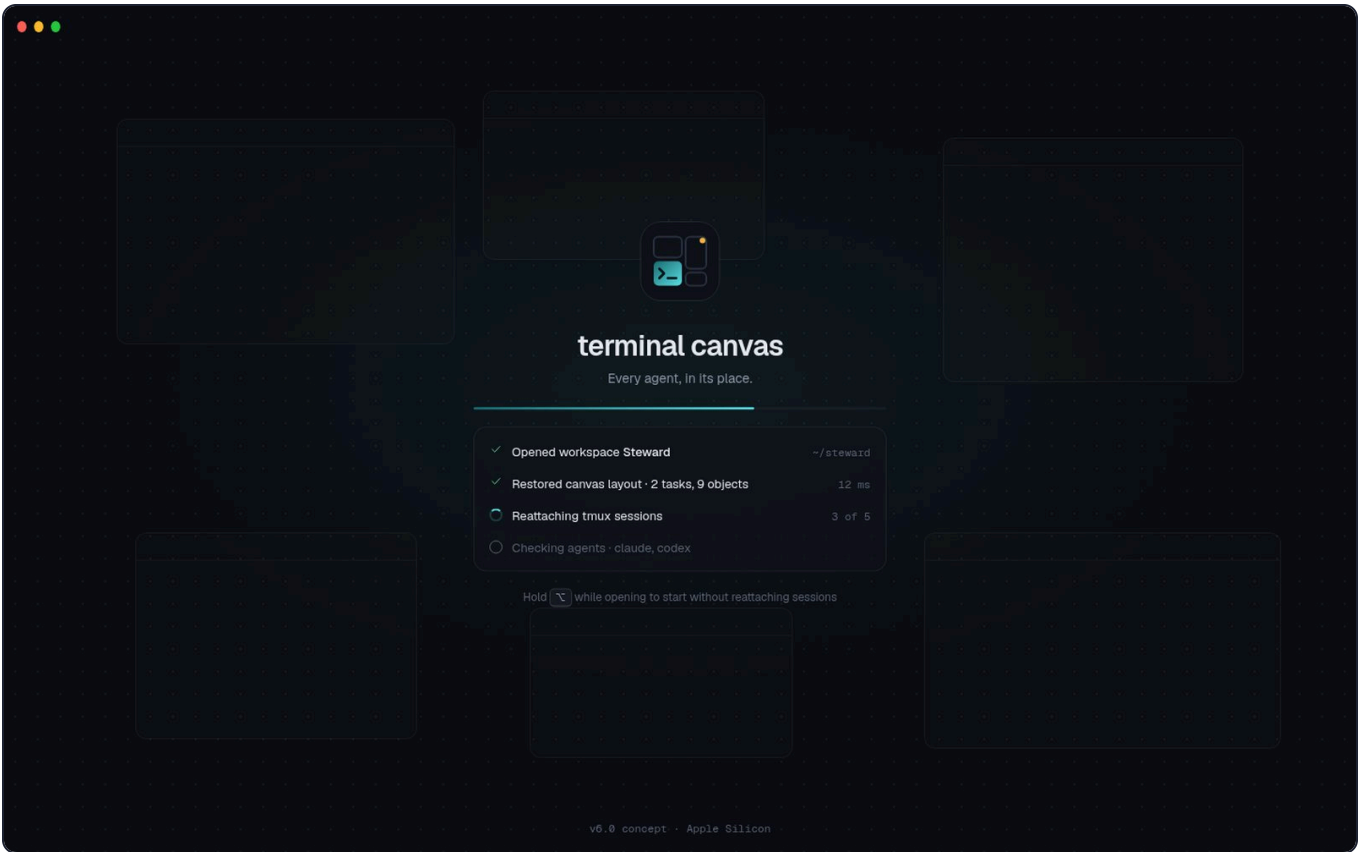
lead, from redesign/main at tag rd-foundations

```
scripts/redesign/lane.sh l-a-launch      L-A rd-foundations
scripts/redesign/lane.sh l-b-workspace  L-B rd-foundations
scripts/redesign/lane.sh l-d-sessions    L-D rd-foundations
scripts/redesign/lane.sh l-e-keyboard    L-E rd-foundations
# In each window: the standard kickoff (§6), then that lane's prompt below.
# Merge order for wave 2a: L-B → L-A → L-E → L-D (each rebases after the previous merge).
```

Lane L-A · 01 Launch/restore, 02 Onboarding, 03 Empty canvas

MILESTONES	BRANCH	WAVE	MOCKUPS
M439 · M440 · M441	rd/l-a-launch	2a (parallel)	01, 02, 03

01 · Launch and restore (M439)



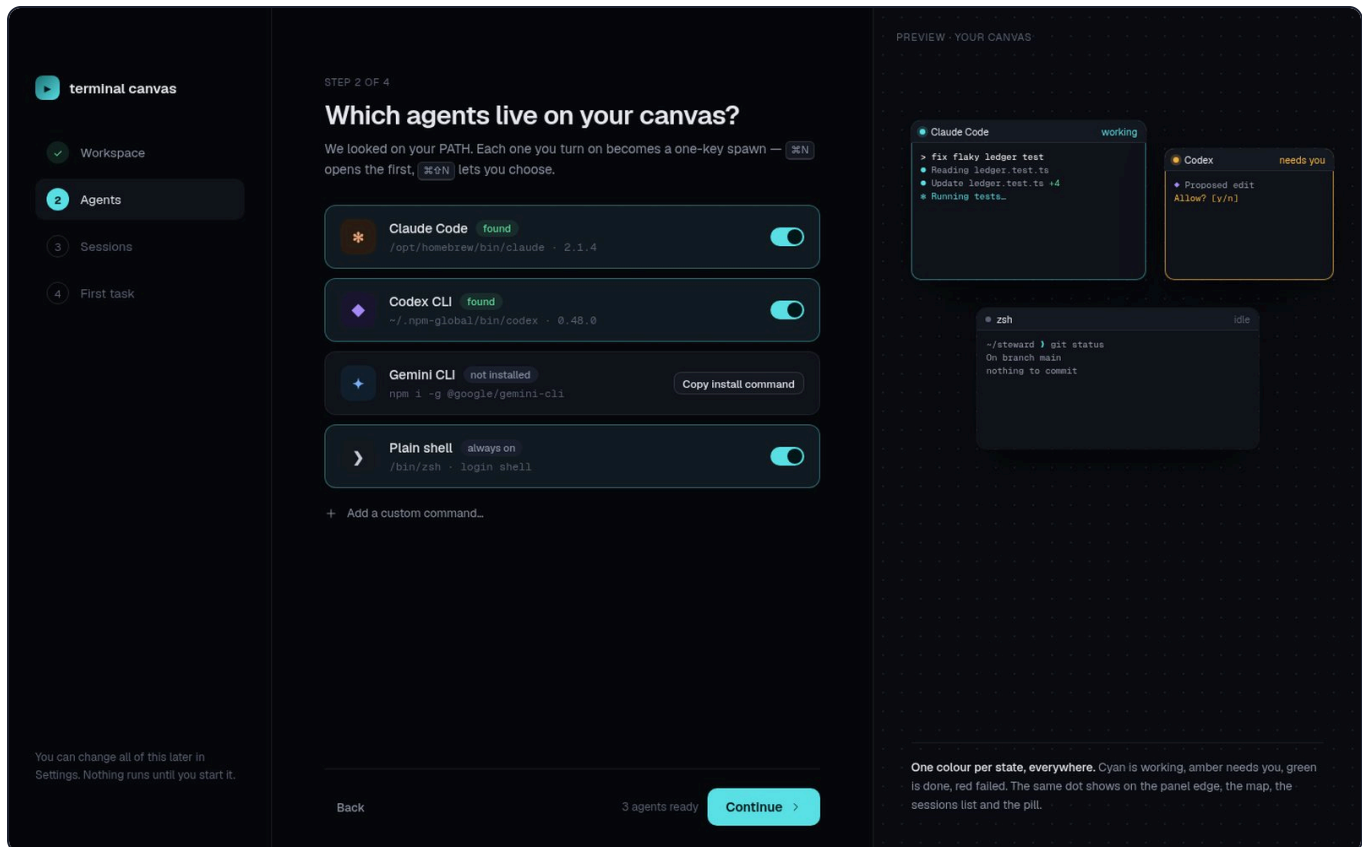
01-splash.png. Brand mark and "Every agent, in its place." An honest restore checklist (workspace · layout · reattaching tmux 3 of 5 · agent check) over the ghosted layout. Hint: hold `⌘` to skip reattaching.

Goal. Launch tells the truth about restore progress, and the person can step out of reattaching.

Files likely touched. `src/renderer/canvas/StartupSplash.tsx`, `canvas/splash.ts`, `session/boot-issues.ts`, `src/shared/resume-summary.ts`, `src/main/bootstrap/*` (emit restore progress on a channel in the pre-cut ipc slot, e.g. `boot:progress`), `src/main/tmux-probe.ts`, `scripts/shot-scenes/rd-l-a.cjs`.

- Four lines driven by real events: workspace opened (name and path in mono), layout restored ("2 tasks, 9 objects", real counts), "Reattaching tmux sessions N of M" (live count), and "Checking agents · claudex, codex". Each line is pending, in progress or done. No line shows progress it hasn't measured.
- The ghosted canvas behind the card is the real layout's skeleton at its real positions, at low opacity.
- Holding `⌘` while the splash shows skips the remaining reattaches. Those panels come up `asleep` (dormant), never killed. A check proves the pane pids survive (the `keep-on-quit` shape).
- A failed step goes to the 09 states instead of a spinner that never ends. Reduced motion means no breathing. The splash leaves as soon as restore is done, with no minimum display time.

02 · First run: agents (M440)



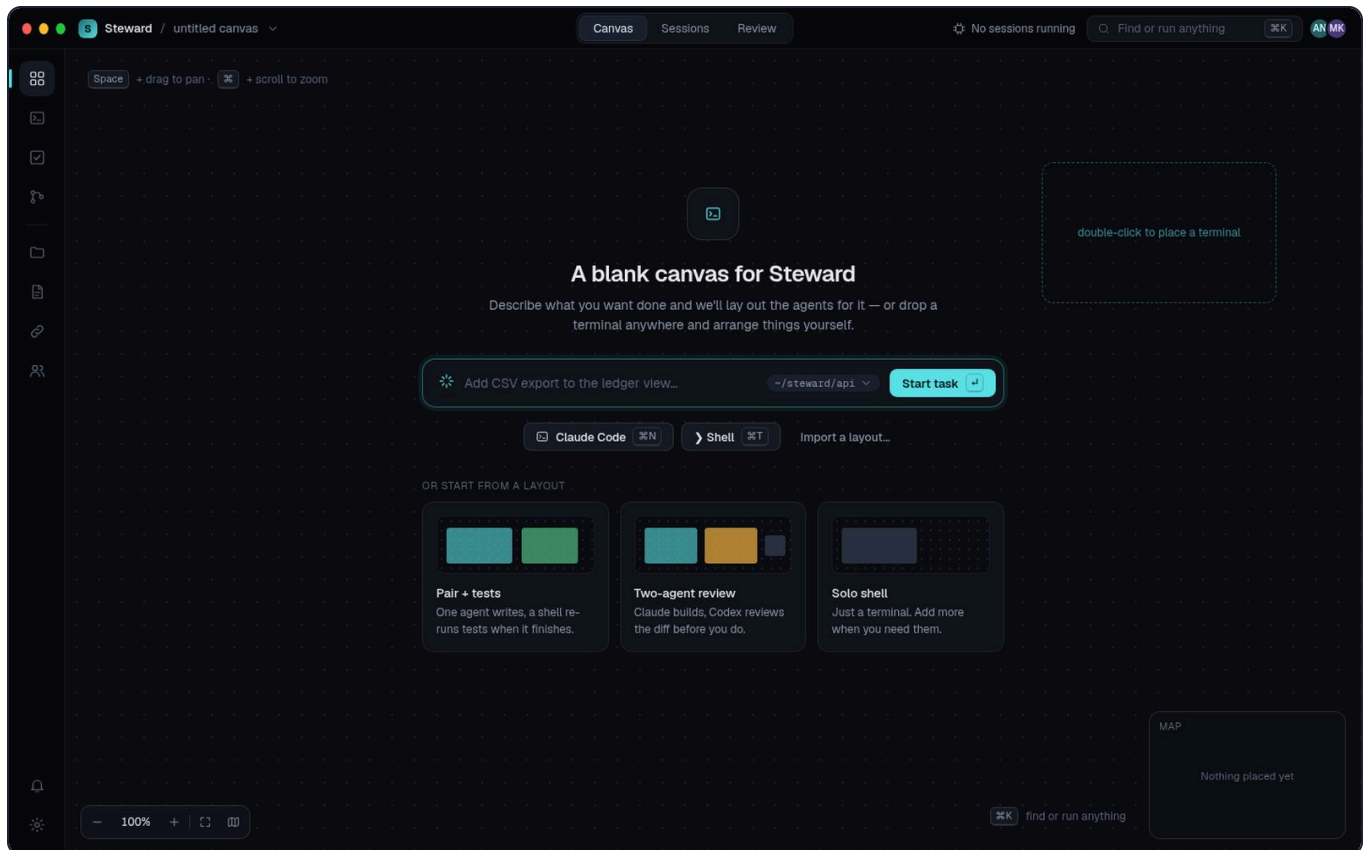
02-onboarding.png. Step 2 of 4 (Workspace → Agents → Sessions → First task). Agents found on PATH with toggles. A missing tool offers its install command. A live preview teaches the state colours.

Goal. First run sets up real agents and teaches the colour system before the first spawn. Nothing runs during onboarding.

Files likely touched. `src/shared/onboarding.ts`, `shared/first-run.ts`, `shared/agent-backends.ts`, `src/main/env-report.ts` + `shared/env-report.ts` (PATH discovery and versions), `src/main/presets.ts` (enabled agents become presets; the first is `⌘N`), new `src/renderer/onboarding/*`, `scripts/verify-onboarding.cjs` / `verify-first-run.cjs` (extend).

- Four steps with a left step rail. Step 2's title is "Which agents live on your canvas?" Each row shows the agent, "found" or "not installed", the resolved path and version in mono, and a toggle. Discovery uses the login-shell PATH that main already resolves, with a timeout per binary.
- A missing agent offers **Copy install command**. It copies and never executes. "Plain shell" is always on. "Add a custom command..." opens the preset editor.
- The right-hand preview renders real panel frames in working, needs-you and idle tones, from the F1 tokens, with the "One colour per state, everywhere..." caption.
- There is one primary control (**Continue**), a "2 agents ready" count that never reads 0 (with none it says "Turn one on to continue"), and the footer "You can change all of this later in Settings. Nothing runs until you start it."
- Step 3 (Sessions) sets tmux persistence ("survive quit"). Step 4 (First task) hands off to the existing New task sheet (`palette/StartWorkSheet.tsx`).

03 · Empty canvas (M441)



03-empty-state.png. Says what the canvas is for, with one primary verb (describe a task → Start task). Quick spawns, three starter layouts, a ghost drop target, navigation hints, and the minimap's "Nothing placed yet".

Files likely touched. `src/renderer/shell/EmptyState.tsx`, `src/shared/empty-states.ts`, `canvas/Launcher.tsx`, `FirstTaskHint.tsx`, `hints.ts`, `palette/start-work.ts`, `shared/lineups.ts` / `templates.ts` (starter layouts). The minimap sentence lives in `empty-states.ts`, and L-B's `MinimapOverlay` reads it.

- Title "A blank canvas for <workspace>" and one sentence of purpose. The task field has a repo chip in mono and the single filled **Start task** ↵.
- Quick spawns: Claude Code ⌘N · Shell ⌘T · Import a layout.... Three starter layouts (Pair + tests, Two-agent review, Solo shell), each one sentence and inert until clicked.

- ```
verify: rail empty.2).
```

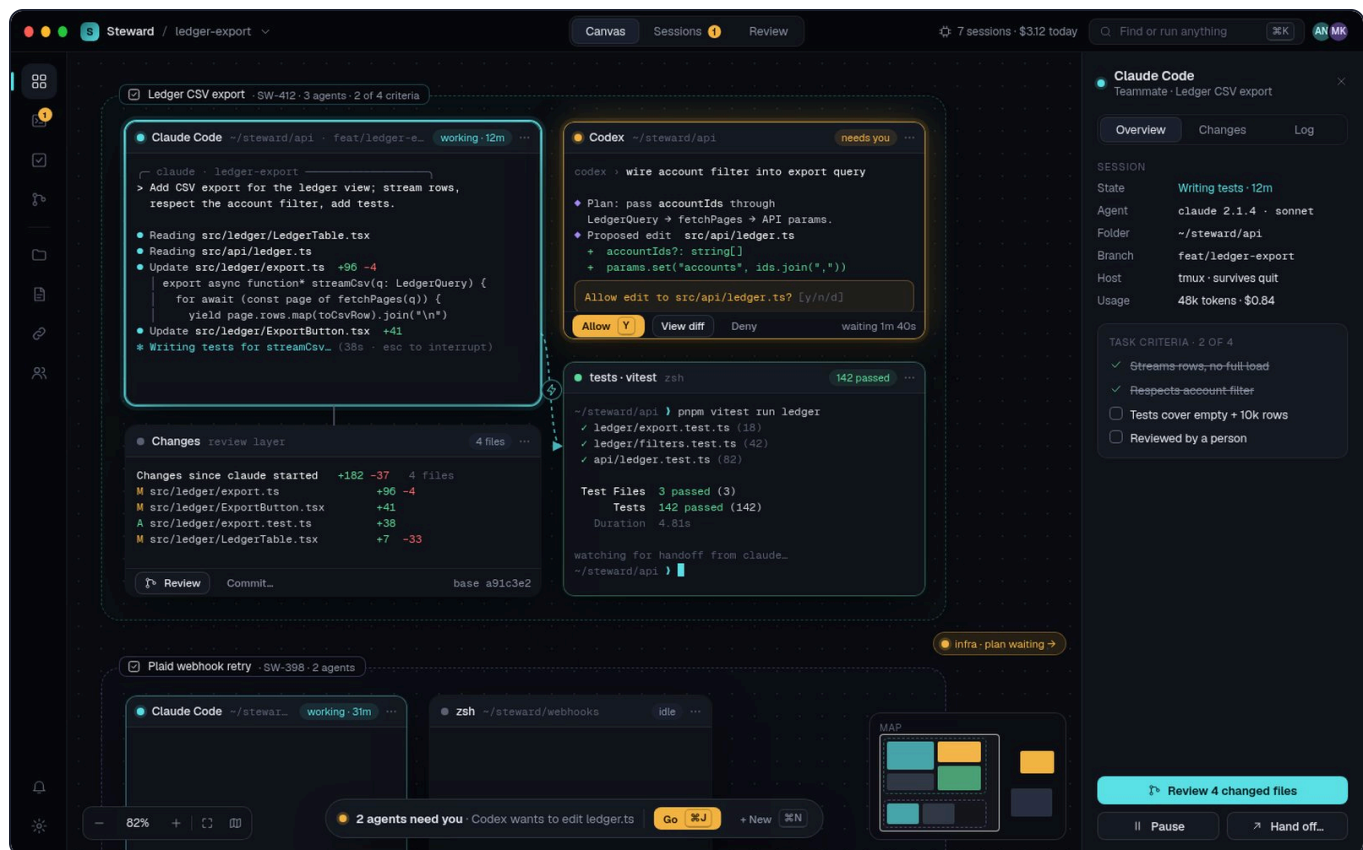
after the standard kickoff

gemini missing) and rd-empty, each with its mockup as reference.

## Lane L-B · 04 Main workspace, then 05 Create and arrange

|                           |                            |                                 |                   |
|---------------------------|----------------------------|---------------------------------|-------------------|
| MILESTONES<br>M442 → M443 | BRANCH<br>rd/l-b-workspace | WAVE<br>2a ·<br>OWNS CANVAS.TSX | MOCKUPS<br>04, 05 |
|---------------------------|----------------------------|---------------------------------|-------------------|

#### 04 - Main workspace, Work tier (M442)



**04-main-workspace.png.** A task region holding Claude (selected), Codex (needs you, with inline Allow / View diff / Deny), a test watcher fed by a handoff edge, and a Changes review card. A second task peeks in below, an off-screen pip points at infra, and the inspector shows the session, the task criteria and one primary action.

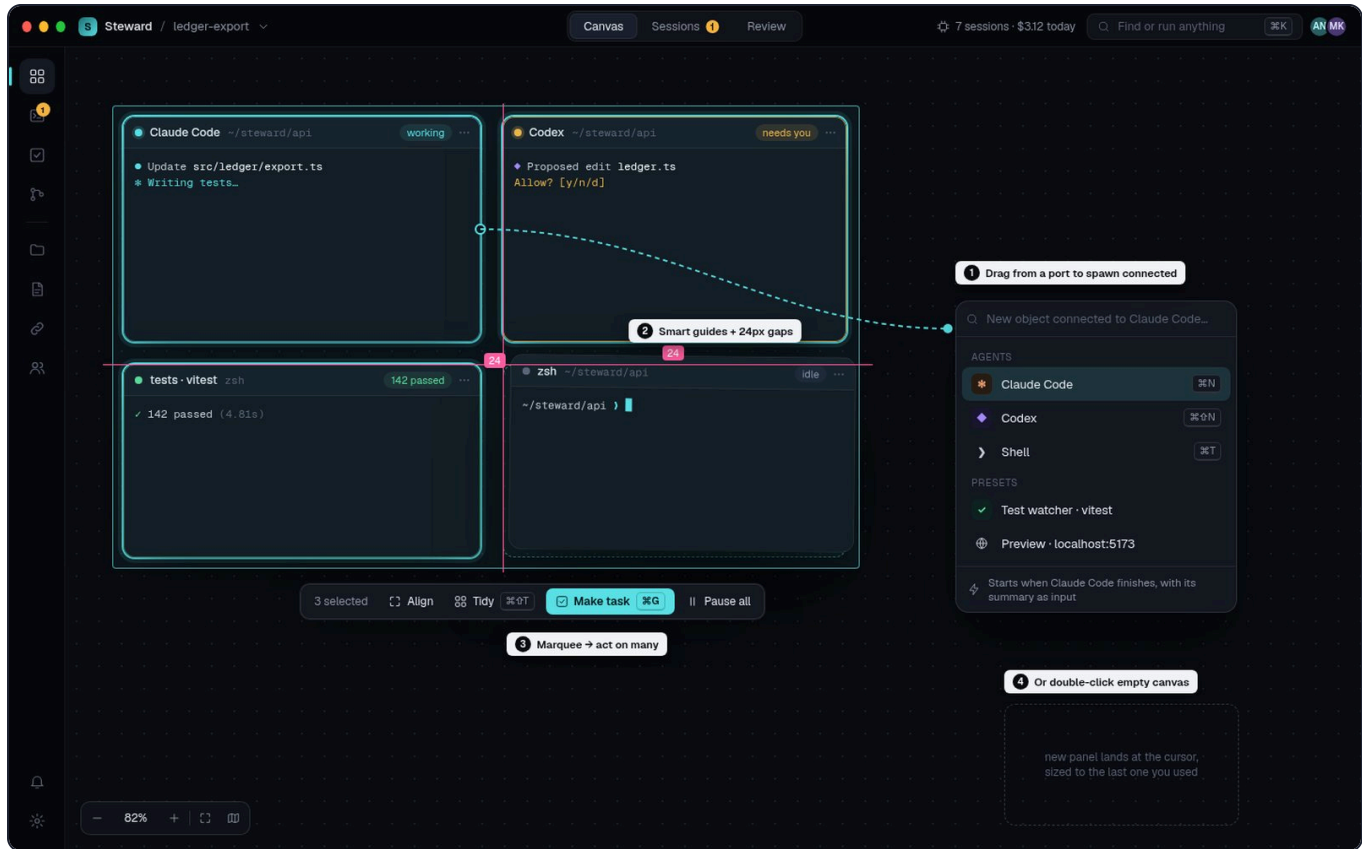
**Goal.** The calm Work-tier canvas. The chrome steps back, task regions are visible, and only state is loud.



| Element            | Spec (from O4 + CONCEPT.md)                                                                                                                                                                                              | Where                                                                                                   |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Task region        | Dashed 1px territory, 20px radius, 24px padding. The label chip reads "Ledger CSV export · SW-412 · 3 agents · 2 of 4 criteria".                                                                                         | <code>TaskClusterLayer.tsx</code> over F2 <code>task-regions.ts</code>                                  |
| Panel frame        | 12px radius. Header: state dot, name, path (mono, faint), branch, and a state pill ("working · 12m"). One resting shadow. Header verbs at opacity 0 → 1.                                                                 | <code>src/renderer/panels/*</code> , styles in <code>rd:L-B</code>                                      |
| Selection          | Per D1: 2px ring + 6px halo, distinct from the working edge.                                                                                                                                                             | styles + panel frame                                                                                    |
| Needs-you panel    | Amber edge + soft glow. An inline request block quotes the agent's question ("Allow edit to <code>src/api/ledger.ts? [y/n/d]</code> ") with <b>Allow</b> (Y, ⌘Y), View diff and Deny, and "waiting 1m 40s".              | <code>shell/ApprovalDetail.tsx</code> , <code>palette/commands/approval-row.ts</code>                   |
| Handoff edge       | A port on the source and a dashed curve to the watcher. It animates only while something crosses it (M233).                                                                                                              | <code>LinkLayer.tsx</code> , <code>AgentLinkLayer.tsx</code> , <code>useHandoff.ts</code>               |
| Changes card       | "Changes since claude started +182 -37 · 4 files", file rows (M/A + counts), Review / Commit..., base sha.                                                                                                               | The existing review node, restyled                                                                      |
| Off-screen pip     | An amber pill at the edge: "infra · plan waiting →". Click flies there.                                                                                                                                                  | <code>EdgeIndicators.tsx</code>                                                                         |
| Inspector          | Overview / Changes / Log. SESSION facts (State, Agent, Folder, Branch, Host "tmux · survives quit", Usage), TASK CRITERIA "2 of 4" checklist, one primary action ("Review 4 changed files"), then Pause and Hand off.... | <code>shell/Inspector.tsx</code> , <code>inspector-fields.ts</code> , <code>inspector-context.ts</code> |
| HUD, minimap, pill | Zoom HUD "82% - + [ ]" at bottom left. Minimap with region outlines, state-coloured blocks and the camera rectangle. The pill comes from F3.                                                                             | <code>CanvasHud.tsx</code> , <code>MinimapOverlay.tsx</code> , <code>minimap.ts</code>                  |

- The `rd-workspace` scene on the `rd-steward` fixture reads as O4 (critic verdict "reads-as" or "close" with no state-colour divergence).
- ⌘Y allows the pending request of the selected panel, or of the queue head when nothing is selected. It routes through the palette executor, so the request's approval door is the same one the inspector uses.
- Restyled, not renamed: every `.panel__*` / `*-node__*` alias is still present (`verify:panels` green). Chromeless terminal chrome stays absolutely positioned. A check measures the xterm well size on header hover and allows Opx change.
- No metric appears on a panel header or card (the age "12m" is a state duration, which D3 allows). Spend appears only in the inspector.

## 05 · Create and arrange (M443)



**05-create-arrange.png.** Drag from a port to spawn a connected object that starts when the source finishes. Smart guides show 24px gaps while dragging. A marquee selection brings up a toolbar: Align, Tidy, Make task ⌘G, Pause all. Double-click places a panel at the cursor.

**Files likely touched.** `canvas/useLinkDraw.ts`, `useHandoff.ts`, `handoff-rules.ts`, `SnapGuides.tsx`, `MarqueeLayer.tsx`, `marquee.ts`, `arrange.ts`, `place-new.ts`, `placement.ts`, `useCanvasPointer.ts`, `palette-actions/arrangement.ts`, `palette-actions/objects.ts`, `palette/SpawnSheet.tsx` (the menu reuses its rows).

- Dropping a port drag on empty canvas opens "New object connected to <source>" with AGENTS (Claude Code ⌘N, Codex, Shell ⌘T) and PRESETS (e.g. Test watcher · vitest, Preview · localhost:5173). The foot reads "Starts when <source> finishes, with its summary as input" (a handoff-on-exit link, M41).
- Smart guides are pink `--guide` and appear only during a drag or resize. Gaps snap to 24px and show their value. The `applyDrag` origin rule and `verify:panels` check 10 stay green.
- Marquee → toolbar "N selected · Align · Tidy ⌘⇧T · Make task ⌘G · Pause all". ⌘G creates a task region from the selection as one undo. Tidy keeps the ⌘⇧T alias (D5).
- Double-clicking empty canvas spawns the default agent at the cursor, sized like the last panel you used. ⌘N and ⌘T also land at the cursor when it's over the canvas. Panels settle with a slight overshoot, and reduced motion removes it.

Lane L-B (M442 then M443): screens 04 and 05 per GUIDE.pdf §7 L-B. You are the ONLY owner of `src/renderer/canvas/Canvas.tsx` in this wave: read `src/renderer/canvas/CLAUDE.md` first (hook order, Deps objects, import direction) and never reorder existing hook calls. lb-scout in parallel on `Canvas.tsx`, `TaskClusterLayer.tsx`, `panels/*`, `Inspector.tsx`, `ApprovalDetail.tsx`, `EdgeIndicators.tsx`, `MinimapOverlay.tsx`, `CanvasHud.tsx`, `LinkLayer.tsx`, `useLinkDraw.ts`, `SnapGuides.tsx`, `MarqueeLayer.tsx`, `place-new.ts`. Build 04 element by element using the table in the guide; write CSS only inside the `rd:L-B` block (plus restyles of existing panel rules, recorded in the ledger). Then 05. Add a stable mount slot in `Canvas.tsx` for wave 2b (a `<TierLayer/>` placeholder and a recovery-overlay slot) so L-C and L-F do not need hook-order edits. Shot scenes: `rd-workspace` (ref 04) and `rd-arrange` (ref 05: freeze mid-drag with guides visible, marquee toolbar up, spawn menu open).

## Lane L-D · 07 Sessions (multi-session management)

| MILESTONE | BRANCH          | WAVE          | MOCKUP |
|-----------|-----------------|---------------|--------|
| M445      | rd/l-d-sessions | 2a (parallel) | 07     |

**07-sessions.png.** The attention queue (approve, triage a failure), a table grouped by task with sparklines, run time, cost and last line, a bulk bar for three selected rows, and a detail pane with a live tail, a reply box, persistence and change stats.

**Goal.** A peer view for fast triage across all sessions. It reads the same state as the canvas, and each view links to the other.

**Files likely touched.** New `src/renderer/sessions/`: `SessionsView.tsx` (mounted in F3's `SessionsHost`), `sessions-model.ts` (pure: rows, grouping, sorting, bulk eligibility), `AttentionCards.tsx`, `SessionsTable.tsx`, `Sparkline.tsx` (inline SVG, because the recharts door is confined to two shell charts), `SessionDetail.tsx`. It

reads `session/session-registry.ts`, `live-session-store.ts`, `last-line-store.ts`, `usage-store.ts`, `machine-series.ts`, and `scrollback:tail`.

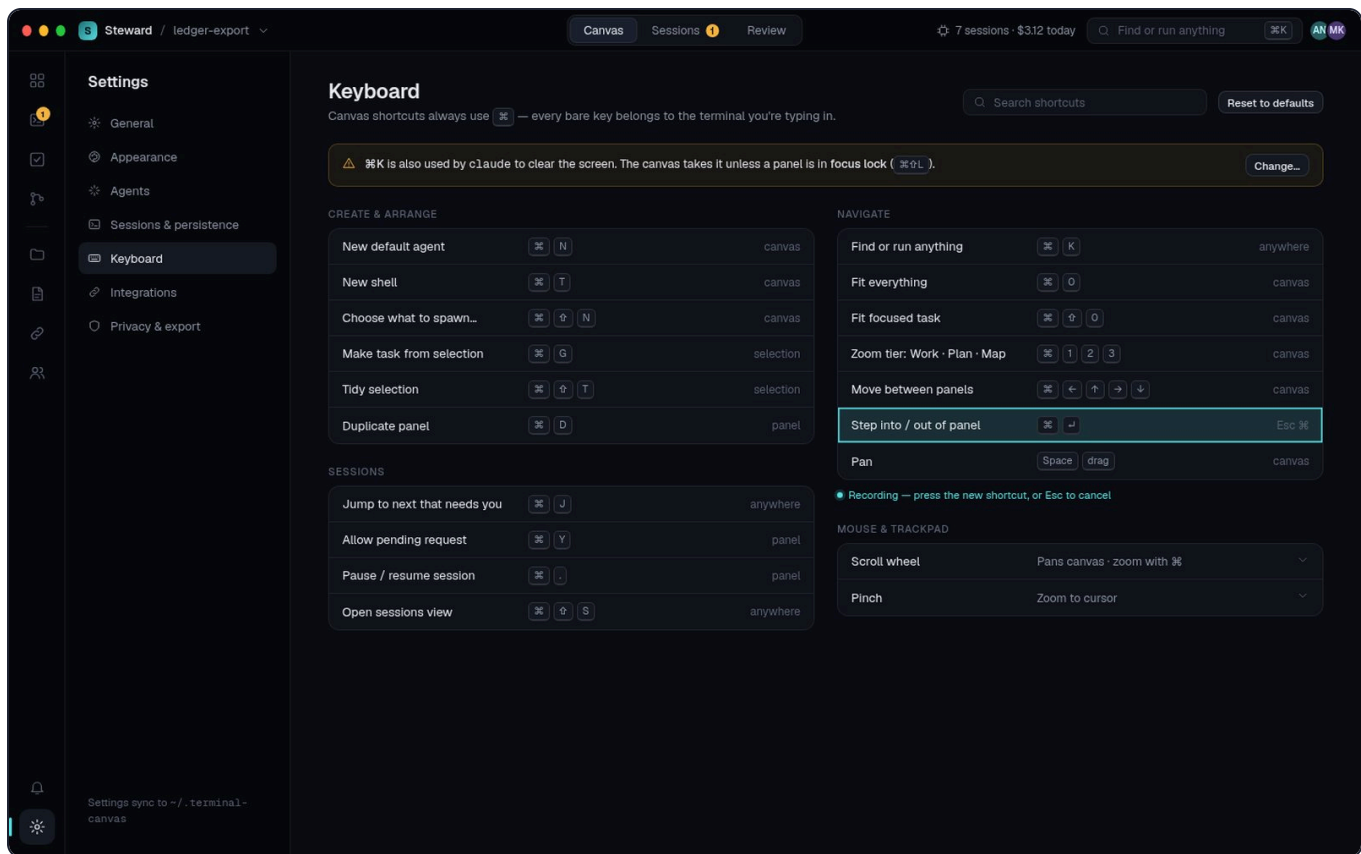
- Header: "Sessions · 7 live · 2 dormant" (never "0 dormant": the part is dropped when zero), Group: Task, All states, and one primary + **New session**. ⌘⇧S opens it.
- Attention cards render `useAttentionQueue` items in queue order with the same sentences as the canvas. Approval: Allow / Diff / Deny. **Shell prompt: Open on canvas / Snooze 10m only, with no inline answer**, keeping the rule that a stray Enter must never run a command. Failure: Restart / Read log.
- The table is grouped by task region with columns Session · Agent · Folder · Branch · State (F1 pill) · Activity (sparkline in the state colour) · Run · Cost · Last line (mono, ellipsised). The bulk bar shows "N selected · Pause · Restart · Move to task... · End", and End asks for confirmation through main's dialog.
- The detail pane shows the live tail (scrollback tail plus a subscription), "Send to this session..." using **paste semantics** and an explicit Send (never raw `write()`, per the Jira gotcha), and facts (Started, Survives "reload and quit (tmux)", Tokens, Changes). Pause · Detach · End session. "Show on canvas" flies the 2D camera to the panel and selects it.
- Metrics here follow D3. A check scopes `metrics.1` so canvas surfaces stay metric-free.

L-D · LANE-SPECIFIC PROMPTafter the standard kickoff

Lane L-D (M445): the Sessions view, screen 07, per GUIDE.pdf §7 L-D and DECISIONS.md D3. Everything lives in new files under `src/renderer/sessions/` mounted into F3's `SessionsHost`; you own no existing canvas file. lb-scout on `session-registry.ts`, `live-session-store.ts`, `last-line-store.ts`, `usage-store.ts`, `machine-series.ts`, `scrollback` IPC and `shell/decision-inbox.ts`. Pure model first (`sessions-model.ts` + checks). Sparklines as inline SVG reading `state-palette.ts`. Reply box: paste semantics + explicit Send; disabled with its reason for shell sessions. Bulk End goes through main's confirmation. Shot scene `rd-sessions` on `rd-steward` (3 rows selected, Claude Code detail open), ref 07.

Lane L-E · 08 Settings › Keyboard

|           |                 |               |        |
|-----------|-----------------|---------------|--------|
| MILESTONE | BRANCH          | WAVE          | MOCKUP |
| M446      | rd/l-e-keyboard | 2a (parallel) | 08     |



**08-settings-keyboard.png.** The ⌘-gated shortcut map (create, sessions, navigate, mouse). It flags the ⌘K conflict with an agent's own ⌘K, proposes focus lock as the answer, and shows a shortcut being re-recorded inline.

**Files likely touched.** New `src/renderer/settings/` ( `SettingsView.tsx` with the left nav General · Appearance · Agents · Sessions & persistence · Keyboard · Integrations · Privacy & export, `KeyboardPane.tsx`, `recorder.ts` pure), new `src/shared/shortcut-overrides.ts` (merges user overrides over the frozen registry), `src/shared/settings-schema.ts` (a keyboard overrides entry), `canvas/palette-actions/settings.ts` (the "Open Settings" row). Main's menu rebuild already follows `settings:changed`.

- ☐ Header copy: "Canvas shortcuts always use ⌘ — every bare key belongs to the terminal you're typing in." Search shortcuts · Reset to defaults.
- ☐ Groups and scopes come from the registry, with no hand-written list ( `rd-keys.settings.1` compares the rendered rows to `shortcuts.ts` ).
- ☐ Conflict banner: "⌘K is also used by cClaude to clear the screen. The canvas takes it unless a panel is in focus lock (⌘⇧L)." with a Change... action.
- ☐ Re-record inline: "Recording — press the new shortcut, or Esc to cancel." Canvas-scope chords without ⌘ are refused by name. A duplicate within a scope shows the clashing row before saving. Overrides persist and survive a restart, and the menu shows the new accelerator.

#### L-E · LANE-SPECIFIC PROMPT

after the standard kickoff

Lane L-E (M446): Settings > Keyboard, screen 08, per GUIDE.pdf §7 L-E and D4/D5. `shortcuts.ts` is frozen: build `src/shared/shortcut-overrides.ts` (pure merge + validation: ⌘ required in canvas scope, duplicate detection, alias handling) with checks first, then the Settings shell and the Keyboard pane. The other settings panes may be thin hosts over existing settings rows for now (each says what it is for — no empty boxes). Shot scene `rd-settings-keys` (recording "Step into / out of panel"), reference 08.



## WAVE 2A MERGE

The lead merges L-B → L-A → L-E → L-D. The lanes still open rebase after each merge, and the lead runs G3 after each one. When all four are in, tag `rd-wave2a`. Ownership of `Canvas.tsx`, `CanvasHud.tsx` and `MinimapOverlay.tsx` passes to L-C, and the panel frame state variants pass to L-F.

## Wave 2b · launch

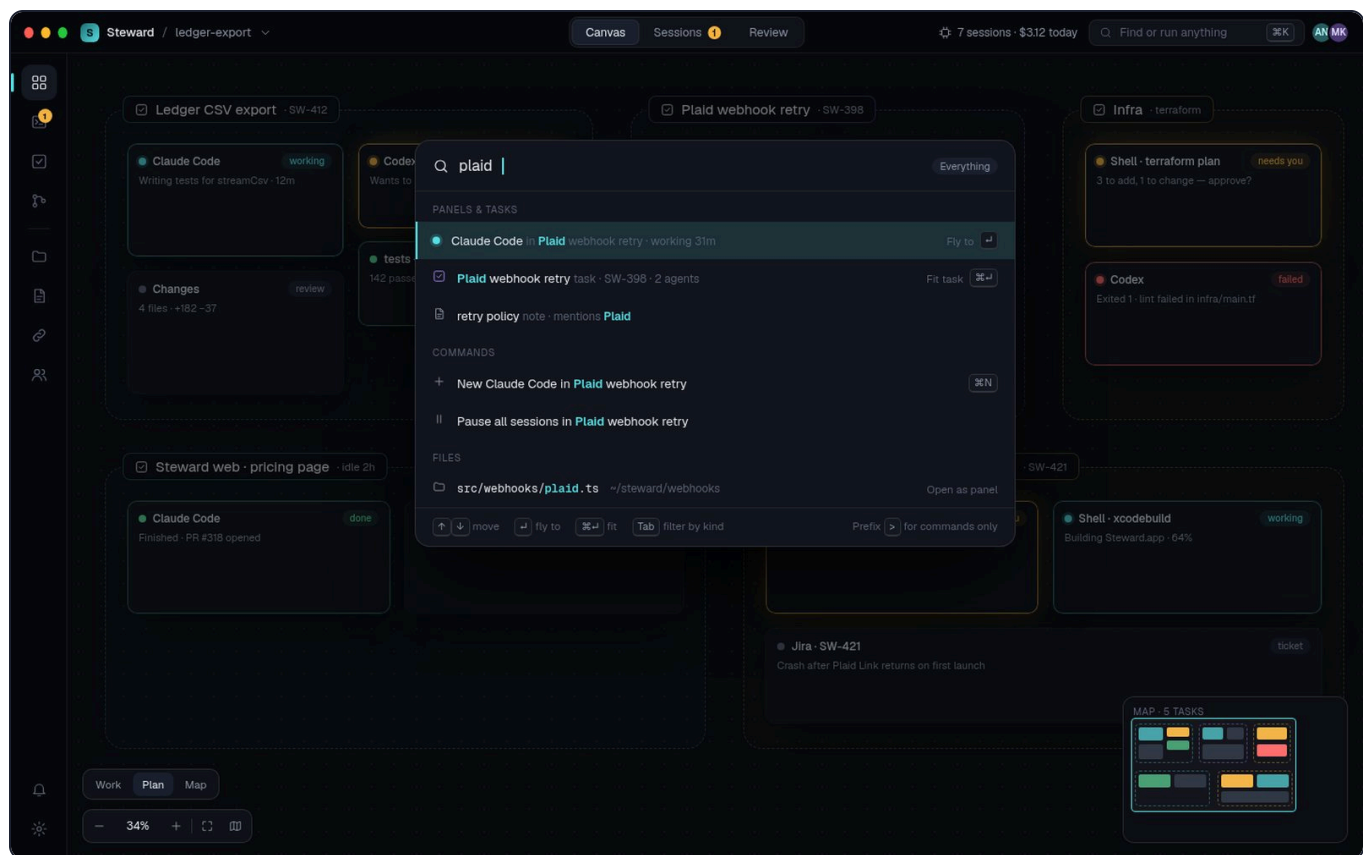
### WAVE 2B · LAUNCH

lead, at tag `rd-wave2a`

```
scripts/redesign/lane.sh l-c-navigate L-C rd-wave2a
scripts/redesign/lane.sh l-f-recovery L-F rd-wave2a
Merge order: L-C → L-F. Then tag rd-canvas and run QA $9.4 part A yourself.
```

## Lane L-C · 06 Navigate: Plan tier and the ⌘K palette

| MILESTONE | BRANCH          | WAVE                 | MOCKUP |
|-----------|-----------------|----------------------|--------|
| M444      | rd/l-c-navigate | 2b · OWNS CANVAS.TSX | 06     |



**06-navigate-palette.png.** Zoomed out to 34%. Panels become status cards, and five task territories show. The palette is grouped (panels & tasks, commands, files) with fly-to and fit-task actions and kind filters. The Work/Plan/Map switch and the minimap sit with the zoom HUD.

**Files likely touched.** `Canvas.tsx` (only in the TierLayer slot L-B left), `canvas/card-detail.ts`, `tier-fade.ts`, `flight.ts`, `useViewport.ts` (only by adding to it; the setter stays private), `CanvasHud.tsx` (Work/Plan/Map switch), `MinimapOverlay.tsx` ("MAP · 5 TASKS"), `palette/Palette.tsx`, `palette-model.ts`, `commands.ts`, `fuzzy.ts`, `palette/commands/*`, `canvas/palette-actions/*`.

- ❑ Tiers use F2's `tierFor`. **Work** ≥70% shows live terminals. **Plan** (~25–70%) shows a card with name, state pill and one status sentence (from the state vocabulary and the agent's last line, e.g. "Writing tests for streamCsv"). **Map** <25% shows only territories, with labels and state dots. Cards come from `lod.ts` `cardIds`, so no session is ended or respawned (a pid-preservation check, the 145/148 shape).
- ❑ ⌘1/2/3 fly to each tier's target scale around the cursor or selection. ⌘0 fits all and ⌘⇧0 fits the selected task. Flights take 220ms with ease-out. Reduced motion lands in one frame (the existing `reduced-motion` scene still passes).
- ❑ The palette is grouped as PANELS & TASKS · COMMANDS · FILES, with match highlighting, a kind filter chip ("Everything"; Tab cycles kinds), and a row action **⇧ fly to** or **⌘⇧ fit task**. Footer key hints. The existing palette checks stay green (49d/49e drill-in, 72c hover stability).
- ❑ ⌘K belongs to the canvas unless focus lock is on (D4). `verify:palette` covers both.

L-C · LANE-SPECIFIC PROMPT

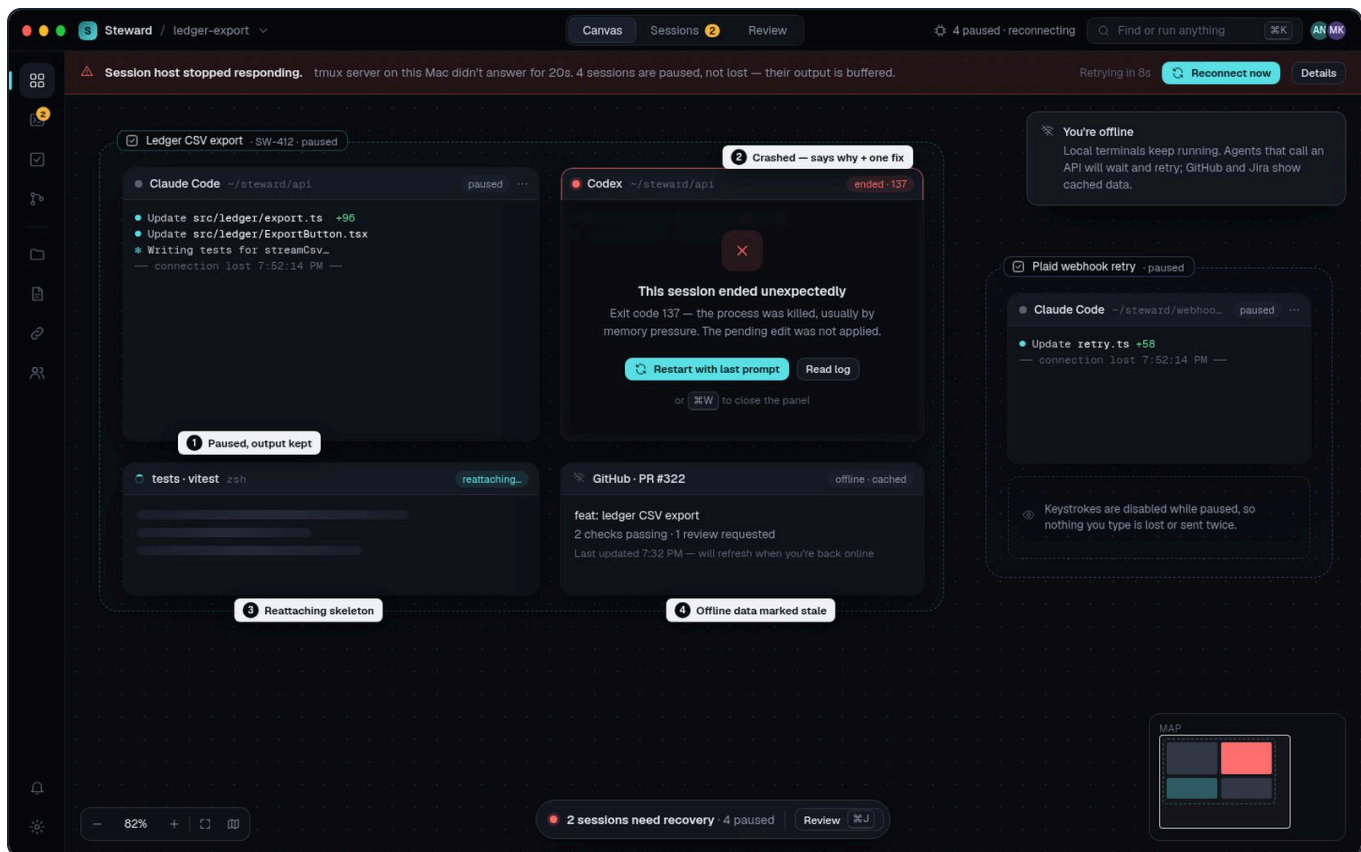
after the standard kickoff

Lane L-C (M444): screen 06 per GUIDE.pdf §7 L-C. lb-scout in parallel on lod.ts, tier-fade.ts, card-detail.ts, flight.ts, useViewport.ts, viewport.ts, CanvasHud.tsx, MinimapOverlay.tsx and the whole palette/ directory (it follows canvas's layering rule). Tiers are presentation only: route cards through lod.ts's cardIds; never change LIVE\_BUDGET, LIVE\_MIN\_SCALE or dormancy precedence; entering Plan/Map releases focus (F2's helper). Mount only in L-B's TierLayer slot. Add pid-preservation checks for tier changes. Palette: restyle and regroup without renaming the classes `verify:panels` selects. Shot scenes `rd-plan-palette` (34%, palette open on "plaid", ref 06) and `rd-map` (18%).

## Lane L-F · 09 Errors and disconnected

|           |                 |               |        |
|-----------|-----------------|---------------|--------|
| MILESTONE | BRANCH          | WAVE          | MOCKUP |
| M447      | rd/l-f-recovery | 2b (parallel) | 09     |





**09-error-disconnected.png.** A tmux host-loss banner (paused, not lost, auto-retry). A crashed session (exit 137) explains itself and offers one fix. Reattaching panels show skeletons, GitHub data is marked offline/cached, an offline toast appears, and the pill reads "2 sessions need recovery".

**Files likely touched.** `shell/ResumeBanner.tsx`, `JobRecoveryNotice.tsx`, `ReopenNotice.tsx`, `shell/toast.ts` (the sonner door), `src/main/session-backend.ts`, `tmux-probe.ts`, `job-recovery.ts`, `last-exit.ts`, `session/live-session-store.ts`, panel frame state variants (paused, reattaching, crashed) in `panels/*` within `id:L-F`, the GitHub/Jira work panels (an offline/cached marker), and new `src/shared/exit-explain.ts` (pure: 137 → "killed, usually by memory pressure", 130, 143, 127, generic).

- Host loss banner: "Session host stopped responding. tmux server on this Mac didn't answer for 20s. 4 sessions are paused, not lost — their output is buffered." with "Retrying in 8s", **Reconnect now** and Details. Affected panels read "paused · output kept". While paused, keystrokes are disabled and the panel says so ("nothing you type is lost or sent twice").
- Crash card: "This session ended unexpectedly" plus the exit-code sentence and what was kept ("The pending edit was not applied"). One primary **Restart with last prompt**, then Read log, with `⌘↵` restarting the panel. Restart in place preserves the panel id (checks 90/92 shape).
- Reattaching panels render skeleton rows, never a blank well. GitHub/Jira data shows "offline · cached · last updated 7:22 PM". The offline toast says what keeps working. The pill reads "2 sessions need recovery · 4 paused · Review" from the attention queue's `recovery` items.
- A test kills the verify tmux server mid-session. Panels go paused and then recover after the retry, with the same pane pids where tmux survived and honest "ended" states where it didn't.

Lane L-F (M447): screen 09 per GUIDE.pdf §7 L-F. lb-scout in parallel on session-backend.ts, tmux-probe.ts, job-recovery.ts, last-exit.ts, live-session-store.ts, ResumeBanner.tsx, JobRecoveryNotice.tsx, toast.ts, and the docs/load-bearing.md entries for quit/keep/reattach. Pure first: src/shared/exit-explain.ts and a recovery-state reducer with checks. Then the main-side host-loss detection (no new polling loop if tmux-probe already has one), the banner, the frame variants inside rd:L-F, and the recovery items feeding useAttentionQueue (F2 contract; if the contract lacks a field, requests.md). The toast is for what is FINISHED; outstanding recovery stays in the attention system. Shot scene rd-recovery on a fixture with a host loss + exit 137, ref 09.

# The 3D World revamp, 10–14

"The World is the canvas, stood up." Same layout (canvas px ÷ 100 = floor units), same colours, same zoom tiers, same attention queue, same chrome. Phase 3 starts from `rd-canvas`, so the 2D foundations it mirrors already exist. Every World lane runs the `world-guard` subagent in G2.

| Wave       | Lanes                                                              | Hotspot owner                                                               | Merge order → tag                  |
|------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------|------------------------------------|
| 3a · SEQ   | W0 night studio + one colour                                       | W0 owns WorldView.tsx, WorldRobot.tsx                                       | W0 → <code>rd-w0</code>            |
| 3b · PAR×3 | W1 transition (10) · W2 room (11) · W3 overview, camera, time (14) | W2 owns WorldView.tsx. W1 owns the Canvas.tsx host seam and WorldStage.tsx. | W2 → W1 → W3 → <code>rd-w3b</code> |
| 3c · PAR×2 | W4 focus (12) · W6 fallback + doors                                | W4 owns WorldView.tsx                                                       | W4 → W6 → <code>rd-w3c</code>      |
| 3d · SEQ   | W5 attention flight (13)                                           | W5 owns WorldView.tsx and WorldFocusSheet.tsx                               | W5 → <code>rd-world</code>         |

## W0 · Night studio and one colour system in the room

| MILESTONE | BRANCH      | WAVE            | MOCKUPS    |
|-----------|-------------|-----------------|------------|
| M448      | rd/w0-night | 3a (sequential) | 11, 12, 13 |

**Goal.** Reverse M415/M416. The room is dark (Obsidian ink), robot shells are neutral, and state is drawn only in the eyes, antenna, floor ring and desk screen, using the canvas colours. Trims are dim neutral, so only state glows, and amber is the only vertical beacon.

**Files likely touched.** `world/world-palette.ts` (replace `STUDIO` with `NIGHT`; `ROBOT_TINTS` become chest-light identity hues only), `world-gloss.ts`, `WorldRobot.tsx` (materials and the four state channels), `world-bloom.ts` / `WorldBloom.tsx` (bloom threshold so only emissive state parts bloom), `WorldCard.tsx` + `WorldCardBody.tsx` (badge colours), `world-roster.ts` (failed and idle agents stay in the room), `scripts/verify-world.cjs` (extend).

- ☐ `rd-world.parity.1`: every state colour in `world/*` is read from `src/shared/state-palette.ts`. A literal state hex in `world/` fails the check, and the F1 exemption list is now empty.
- ☐ `rd-world.identity.1`: no identity hue is within ΔE 20 of any of the five state colours (so no orange that reads as amber, no teal that reads as cyan).
- ☐ A WORKING badge is cyan and a done one is green, matching the canvas. M415's green-for-working is gone, and "QUIET" is no longer red.
- ☐ An errored agent stays at its desk, slumped, with a red ring. Agents never walk to a central table (the M422 meeting table is retired, and the plan whiteboard stays).
- ☐ Bloom is on only for state emitters. The ground matches `--s-0` ink, so the room and the chrome read as one material.

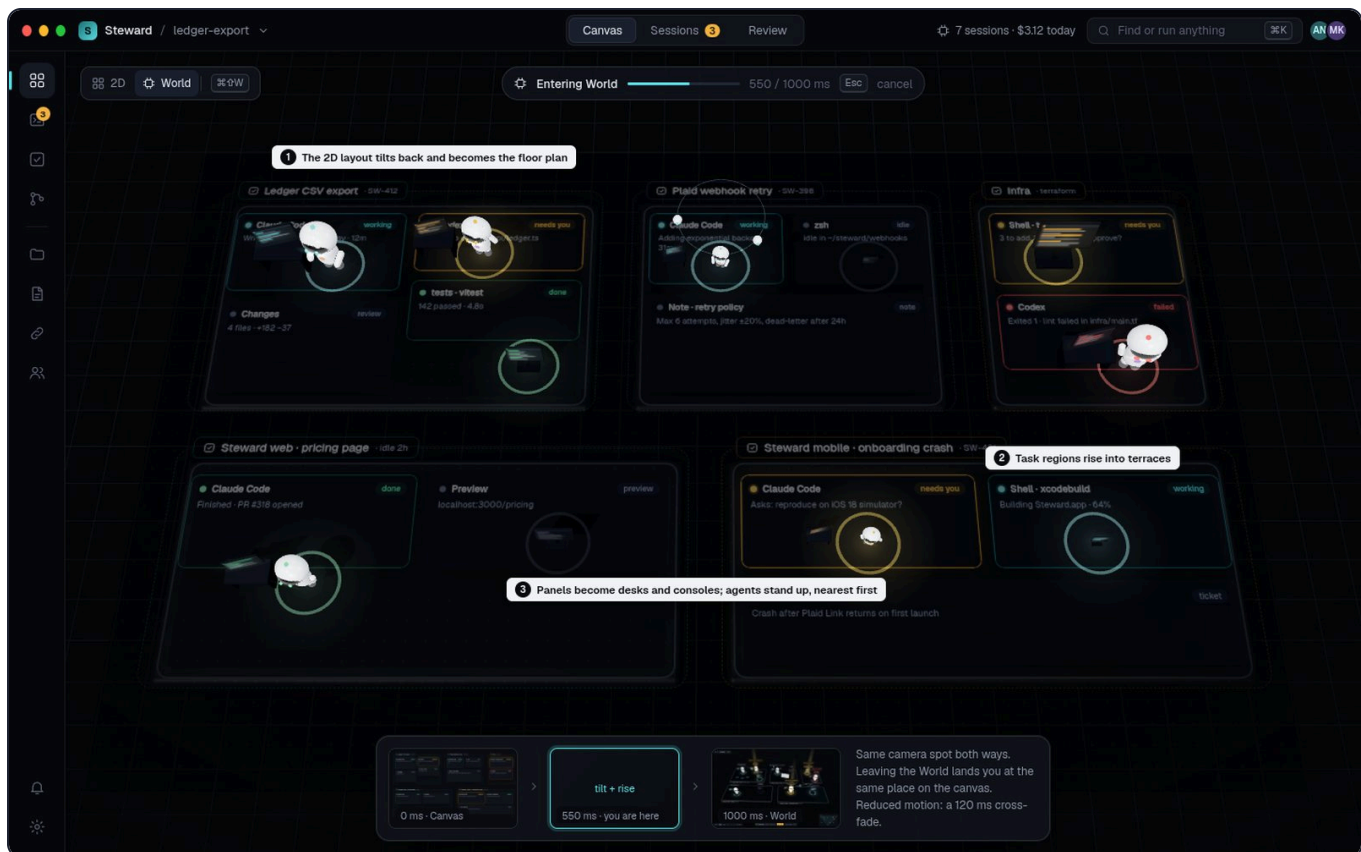
Lane W0 (M448): the night studio and one colour system in the World, per GUIDE.pdf §8 W0. Read the build logs docs/build-log/m412-world-scene.md through m432-m434-world-crosslink.md (use parallel Explore/lb-scout subagents; return only decisions and recorded gaps), src/renderer/CLAUDE.md's three/R3F rows, and world-palette.ts, world-gloss.ts, WorldRobot.tsx, world-bloom.ts, WorldCard.tsx, world-roster.ts. Replace STUDIO with NIGHT; make shells neutral; route every state colour through state-palette.ts; identity hues only on a chest light and never near a state hue; keep failed and idle agents in the room. Keep every world.door.\* / world.bloom.\* check green and the pure modules three-free. Shot scene rd-world-night (Plan tier, rd-steward sim feed), reference 11. Run the world-guard subagent before G2.

## Wave 3b · launch

```
scripts/redesign/lane.sh w1-transition W1 rd-w0
scripts/redesign/lane.sh w2-room W2 rd-w0
scripts/redesign/lane.sh w3-overview W3 rd-w0
W2 owns WorldView.tsx: W1 and W3 export hooks and components; W2 (or the lead at merge) mounts them.
```

## W1 · 10 Canvas → World transition

| MILESTONE | BRANCH           | WAVE          | MOCKUP |
|-----------|------------------|---------------|--------|
| M449      | rd/w1-transition | 3b (parallel) | 10     |



**10-world-transition.png** (shown at 550 / 1000 ms). The 2D plan tilts back into the floor, terraces rise, and robots pop up nearest-first. A cancel chip, the **2D | World** lens and a filmstrip explain the move. Reduced motion uses a 120 ms cross-fade.

**Files likely touched.** `world/world-transition.ts` (still one pure clock: add the plan tilt, terrace rise and nearest-to-focus stagger; reduced motion becomes a 120 ms cross-fade instead of a snap), `WorldStage.tsx`, `world-toggle.ts` (`⌘⇧W` via the registry), the Canvas host seam (opacity and tilt of the 2D layer), a new `world/WorldLens.tsx` (the **2D | World** `⌘⇧W` segmented control at the canvas's top left), and a new `world/plan-floor.ts` (draws the floor texture **from layout data**, not from a screenshot).

- ☐ The move takes 1000 ms with `easeInOutCubic`, and every reader samples the same `WorldTransition`. The 2D plan and the 3D floor coincide at the hand-off frame, because both use `world-space.ts` (the max misalignment check passes within 2 px at 1440×900).
- ☐ Terraces rise from the floor, and robots pop nearest-first **from the camera target**, with `popDelays` measured from the target, not from the room centre.
- ☐ The cancel chip reads "Entering World · Esc cancel" with progress. Esc mid-move reverses at the same speed (the existing ramp property). Toggling back lands the 2D viewport on the camera's target spot (`cameraToViewport`).
- ☐ Reduced motion: a 120 ms cross-fade with no dolly, tilt or pop. The canvas stays mounted throughout, terminals keep running (a pid check), and keys never reach the hidden canvas.
- ☐ The old TopBar "World view" button is removed through a `requests.md` item to the lead (TopBar is frozen). If `verify:world / verify:panels` select `.shell__world-toggle`, the lens keeps that class.



W1 · LANE-SPECIFIC PROMPT

after the standard kickoff (RD\_LANE=W1)

Lane W1 (M449): screen 10, the transition, per GUIDE.pdf §8 W1. world-transition.ts stays the ONE pure clock (read its header); extend it with plan-tilt, terrace-rise and target-relative pop curves, and replace the reduced-motion snap with a 120 ms cross-fade – checks first in verify:rd-w1 and verify:world. Build plan-floor.ts to paint the floor texture from the layout (regions + panel rects via world-space.ts), never from a captured PNG. WorldLens.tsx replaces the TopBar button (file the TopBar removal in requests.md). Export everything W2 must mount; touch WorldView.tsx only through W2. Shot scenes rd-world-transition frozen at 550 ms (ref 10) and rd-world-transition-rm (reduced motion, mid-fade).

W2 · 11 The World, Plan tier

|           |            |                         |        |
|-----------|------------|-------------------------|--------|
| MILESTONE | BRANCH     | WAVE                    | MOCKUP |
| M450      | rd/w2-room | 3b · OWNS WORLDVIEW.TSX | 11     |



11-world-main.png. Five task terraces at their canvas positions. Robots and consoles show state rings, eyes and screens. Amber beacons rise from three waiting agents, and the failed Codex slumps with a red ring. Handoff arcs, file tiles, the plan whiteboard and orbiting sub-agents. The bottom pill and the 2D minimap with the camera wedge are shared with the canvas.

**Files likely touched.** WorldStructure.tsx + world-structure.ts (terraces from task-regions.ts × canvasToFloor), WorldOffice.tsx (desks for agents, consoles for plain shells, at panel positions), WorldPlatform.tsx, WorldProps.tsx, world-scene.ts, world-set.ts, WorldRobot.tsx (poses: typing, raised arm, slump), WorldCard.tsx (density layers), WorldChrome.tsx (retire the "Ask your team" pill, the legend and Fit room, and mount the shared pill, minimap and tier controls), WorldView.tsx, world-perf.ts / world-quality.ts (keep instancing and level of detail).

- ❑ `rd-world.layout.1`: every terrace centre and size equals `canvasToFloor` of its region. Every desk or console sits at its panel's floor point. Dragging a terrace runs the same region-move verb as 2D, with one undo, and the 2D canvas shows the move.
- ❑ State: working = cyan eyes, ring and screen, gently breathing. Needs-you = raised arm, amber ring and an **amber vertical beacon** (the only beacon). Done = green ring. Failed = slumped with a red ring and a red pill ("Exited 1 · lint failed"). Idle = slate.
- ❑ Density: name pills with a state dot at rest. Ask chips only for waiting agents ("Wants to edit ledger.ts", "3 to add · approve?"). Full cards only for the picked agent, which fixes the six-card clutter. Terrace label: "Ledger CSV export · SW-412 · 2/4".
- ❑ Shared chrome: the title bar ( `Canvas` active), the bottom pill "3 need you · 1 failed · 4 working · Go ⌘J · Ask an agent ⌘L", the 2D minimap with a camera wedge, and the Work / Plan / Map switch plus Follow at bottom left. `rd-world.chrome.1` is a source check that these are the same components as 2D.
- ❑ Performance: 20 agents hold the `world-perf.ts` budget. Total WebGL contexts (xterm live budget 8 + World 1) stay well under 16.

#### W2 · LANE-SPECIFIC PROMPT

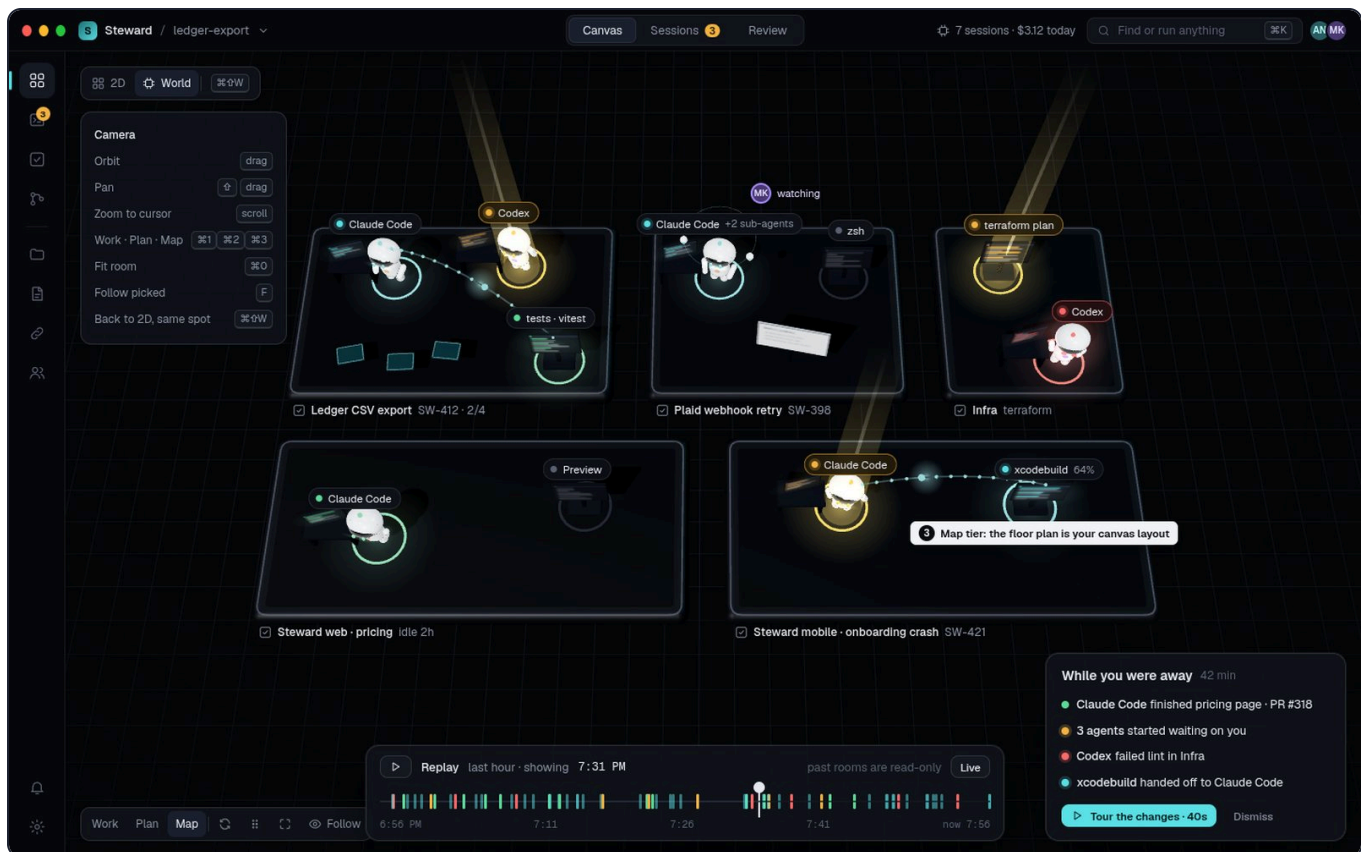
after the standard kickoff (RD\_LANE=W2)

Lane W2 (M450): screen 11 per GUIDE.pdf §8 W2. You own `WorldView.tsx` this wave; W1 and W3 hand you hooks/components to mount – mount them in clearly marked blocks and keep their props stable. `lb-scout` in parallel on `WorldStructure.tsx`, `world-structure.ts`, `WorldOffice.tsx`, `WorldPlatform.tsx`, `WorldProps.tsx`, `world-scene.ts`, `world-set.ts`, `WorldCard.tsx`, `WorldChrome.tsx`, `world-perf.ts` and `world-quality.ts`. Positions come only from `task-regions.ts` + `world-space.ts`. State visuals only from `state-palette.ts`. Replace `WorldChrome`'s own pill/legend/fit with the shared 2D components (the World is a lens inside Canvas, not a fourth tab). Shot scene `rd-world-room` (Plan tier, sim feed with 3 waiting, 1 failed, 4 working), reference 11. `world-guard` before G2.

## W3 · 14 Overview, camera and time

| MILESTONE | BRANCH         | WAVE          | MOCKUP |
|-----------|----------------|---------------|--------|
| M451      | rd/w3-overview | 3b (parallel) | 14     |





**14-world-overview.png.** The Map tier, near top-down: the floor plan is the canvas layout. The camera panel; a replay scrubber with state-coloured ticks (past rooms are read-only); "While you were away" with a 40 s tour; a teammate (MK) watching an agent.

**Files likely touched.** New `world/world-camera.ts` (pure: orbit, pan, zoom-to-cursor, tier poses for  $\mathbb{K}1/2/3$ , fit, follow), a new `world/WorldCameraPanel.tsx`, `WorldTime.tsx` + `world-replay.ts`, `world-away.ts`, `WorldPeers.tsx`, `WorldMinimap.tsx` / `world-minimap.ts` (folded into the shared minimap's camera wedge).

- ☐ The camera panel lists Orbit (drag), Pan ( $\uparrow$  drag), Zoom to cursor (scroll / pinch with `ctrlKey`), Work · Plan · Map  $\mathbb{K}1 \mathbb{K}2 \mathbb{K}3$ , Fit room  $\mathbb{K}0$ , Follow picked F, and Back to 2D at the same spot  $\mathbb{K}\uparrow W$ . Every chord comes from the registry.
- ☐ `rd-world.fit.1`: Fit centres the terrace bounds to within 2% of the viewport, which fixes "Fit room sits off-centre". `rd-world.zoom.1`: zoom-to-cursor keeps the floor point under the cursor fixed (pure check).
- ☐ Replay: "Replay · last hour" with ticks coloured by state-palette and a Live button. A past room is read-only, and every verb is disabled with the reason "past room — go Live to act".
- ☐ "While you were away · 40 min" lists real events from the feed, with **Tour the changes · 40s** and Dismiss. Under reduced motion the tour cuts between shots instead of flying. A teammate's initials chip ("MK watching") sits on the agent they are watching.

### W3 · LANE-SPECIFIC PROMPT

after the standard kickoff (RD\_LANE=W3)

Lane W3 (M451): screen 14 per GUIDE.pdf §8 W3. Pure first: `world-camera.ts` (orbit/pan/zoom-to-cursor/tier poses/fit/follow) with `verify:rd-w3` checks including the off-centre fit regression. Then the camera panel, replay (read-only past: verbs disabled with their reason), away tour and peers. Do not edit `WorldView.tsx` – export `useWorldCamera()` and components for W2 to mount. The minimap becomes the shared 2D minimap with a camera wedge computed by `cameraToViewport`. Shot scene `rd-world-overview` (Map tier, scrubber visible, away card open), reference 14.

# Wave 3c · launch

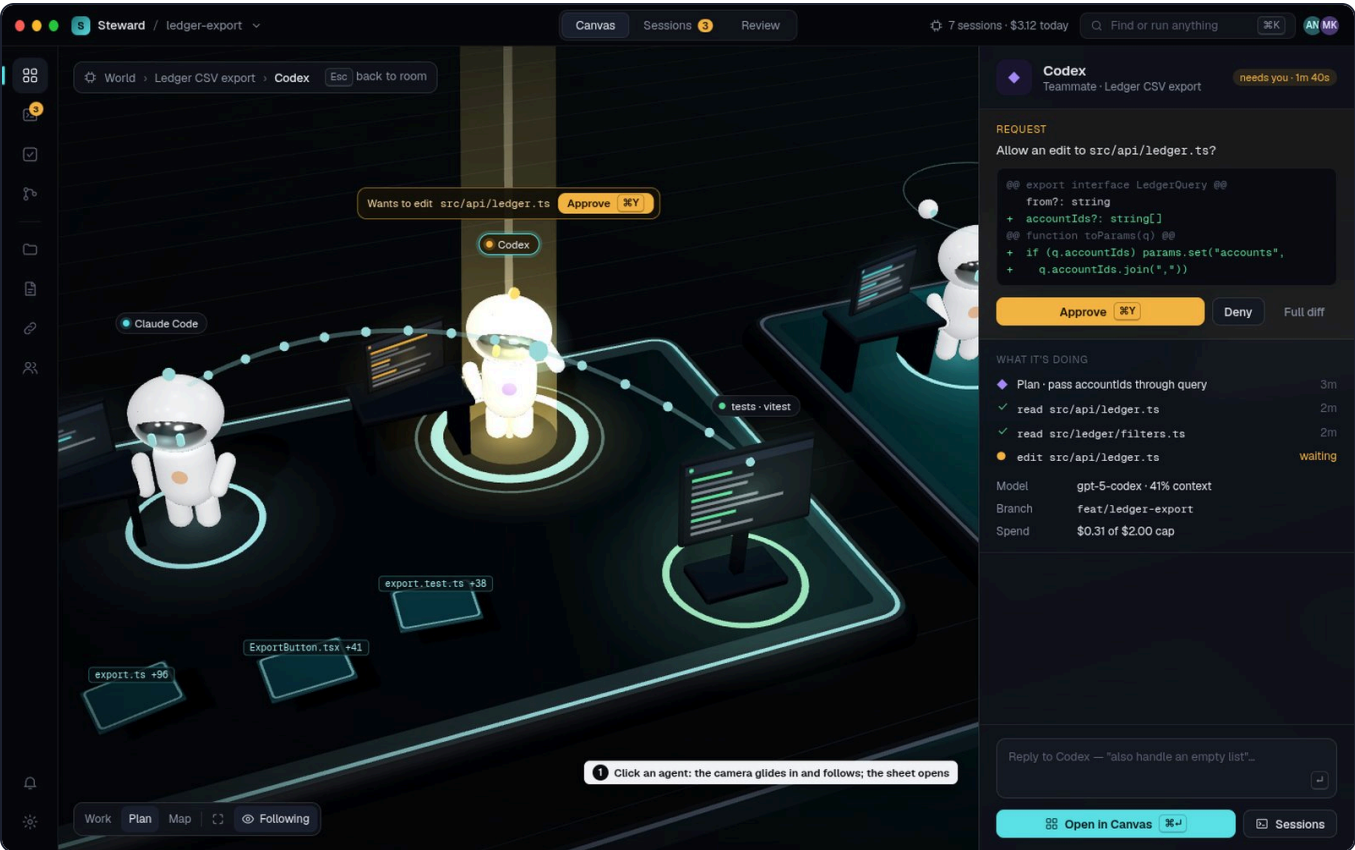
WAVE 3C · LAUNCH

lead, at tag rd-w3b

```
scripts/redesign/lane.sh w4-focus W4 rd-w3b
scripts/redesign/lane.sh w6-doors W6 rd-w3b
```

# W4 · 12 Focus and act

|                   |                       |                                    |              |
|-------------------|-----------------------|------------------------------------|--------------|
| MILESTONE<br>M452 | BRANCH<br>rd/w4-focus | WAVE<br>3c ·<br>OWNS WORLDVIEW.TSX | MOCKUP<br>12 |
|-------------------|-----------------------|------------------------------------|--------------|



**12-world-focus.png.** Picking Codex glides the camera in and turns on Follow. Breadcrumb `World > Ledger CSV export > Codex`. `Esc`. An inline ask chip with Approve ⌘Y. A 400px focus sheet with the diff, Approve / Deny / Full diff, what the agent is doing, facts, a reply composer, and Open in Canvas ⌘⇧ / Sessions.

**Files likely touched.** `world/world-select.ts`, a new `world/WorldFocusSheet.tsx` (plain DOM, imports no three), `WorldCardBody.tsx`, the breadcrumb in `WorldChrome.tsx`, and `WorldView.tsx` (camera glide and follow through W3's `useWorldCamera`).

- Click (or Enter on a keyboard-picked robot) glides the camera in, turns on Follow, and shows the breadcrumb. Esc steps back out to the room, and ⌘Esc goes back to Plan.
- The sheet is 400px wide and has three sections. REQUEST: the question, the diff, and **Approve ⌘Y** (filled while a request is pending) / Deny / Full diff. WHAT IT'S DOING: plan steps with states. Facts: model · context %, branch, spend vs cap (inspector density, allowed by D3). Below them, the reply composer and Open in Canvas ⌘⇧.

Sessions. When nothing is pending, Open in Canvas becomes the filled control, so there is still one primary per surface.

- Approve, Deny and Reply call the **same verbs** as the 2D inspector and Sessions, through the palette executor. The reply is enabled for chat agents ( `agent:send` ) and agent terminals (paste + explicit Send). It is disabled with the reason "answer in its terminal" for plain shells.
- Open in Canvas (⌘⇧ or double-click) runs the reverse transition, lands the 2D camera on the panel, and selects it (M429 behaviour kept).

#### W4 · LANE-SPECIFIC PROMPT

after the standard kickoff (RD\_LANE=W4)

Lane W4 (M452): screen 12 per GUIDE.pdf §8 W4. lb-scout on world-select.ts, WorldCardBody.tsx, WorldChrome.tsx, shell/ApprovalDetail.tsx, palette/commands/approval-row.ts and the M428-M429 build log. The focus sheet is plain DOM and reuses the 2D approval/reply verbs through the palette executor – no second approval path. Camera glide/follow via useWorldCamera. Shot scene rd-world-focus (Codex picked, request pending), reference 12.

## W6 · WebGL fallback and the doors both ways

| MILESTONE | BRANCH      | WAVE          | MOCKUPS                        |
|-----------|-------------|---------------|--------------------------------|
| M454      | rd/w6-doors | 3c (parallel) | 11 (words), 07 (Show in World) |

**Files likely touched.** `world/WorldFlat.tsx` (its import closure stays free of scene files: `world.flat.1`), `WorldStage.tsx` (context-loss → flat; W1's file is free after its merge), `src/renderer/webgl-probe.ts` (read only), `sessions/*` (a "Show in World" row action, in L-D's files, now owned by W6), `canvas/object-verbs.ts` + `palette-actions/objects.ts` ("View in World" for a panel or task).

- No WebGL (probe false), or a `webglcontextlost` event: the flat room appears with the same terraces as tiles, the same state words and colours, the same pill and queue, and the same Approve / Open in Canvas doors. A check forces the probe off and asserts the doors exist.
- "View in World" on any panel or task, and "Show in World" on a Sessions row, land the World camera on that floor point with the agent picked. The Orchestrate cross-links from M432–M434 still work.

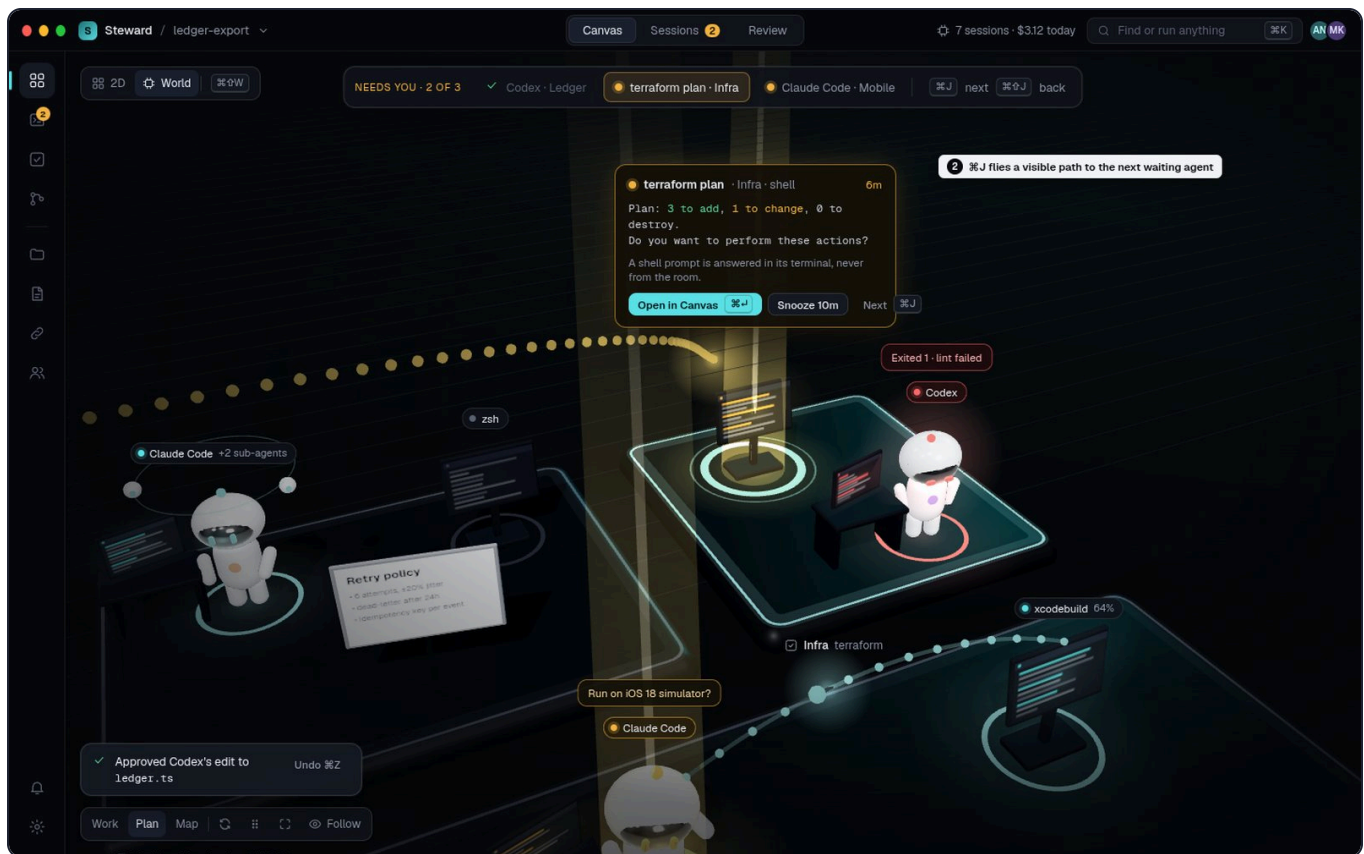
#### W6 · LANE-SPECIFIC PROMPT

after the standard kickoff (RD\_LANE=W6)

Lane W6 (M454): the WebGL fallback and the cross-view doors per GUIDE.pdf §8 W6. Keep WorldFlat's import closure free of scene files (`world.flat.1`). Add context-loss handling in WorldStage that falls back to WorldFlat without unmounting the 2D canvas. Add the doors through existing verb plumbing (`object-verbs/palette` rows), so each door also gets its palette row (V9\_DOORS rule). Shot scene rd-world-flat (probe forced off), with reference 11 for words (not visuals).

## W5 · 13 Attention flight (⌘J)

| MILESTONE | BRANCH       | WAVE                                          | MOCKUP |
|-----------|--------------|-----------------------------------------------|--------|
| M453      | rd/w5-flight | 3d (sequential, last) ·<br>OWNS WORLDVIEW.TSX | 13     |



**13-world-attention.png.** After approving Codex (with Undo), %J flies a visible amber path to the next waiting agent. A queue strip shows Needs you · 2 of 3 with %J / %J. The terraform shell's card says it must be answered in its terminal and offers Open in Canvas.

**Files likely touched.** A new `world/world-flight.ts` (pure: an arc between floor points, duration by distance clamped to roughly 600–1200 ms, and a cut under reduced motion), a new `world/WorldQueueStrip.tsx`, `WorldView.tsx`, `WorldFocusSheet.tsx` (the shell-console variant), and the approval toast via `shell/toast.ts`.

- ☐ %J walks `useAttentionQueue`, the same queue as 2D. The camera flies along a visible amber dotted path to the next item and picks it. %J goes back. The strip reads "NEEDS YOU · 2 of 3" with done ✓ / current / next chips.
- ☐ After Approve, a toast reads "Approved Codex's edit to ledger.ts · Undo". **Undo must be honest:** either approval sends after a short visible hold that Undo cancels, or Undo is offered only where a revert exists (review discard). Pick one in DECISIONS.md, because an agent that already received "y" can't be un-told.
- ☐ A shell console card ("terraform plan · Infra") shows the prompt text and "A shell prompt is answered in its terminal, never from the room." Open in Canvas ⌘J · Snooze 10m · Next. There is no reply field.

#### W5 · LANE-SPECIFIC PROMPT

after the standard kickoff (RD\_LANE=W5)

Lane W5 (M453): screen 13 per GUIDE.pdf §8 W5. Pure world-flight.ts first (path, duration, reduced-motion cut) with checks. %J/%J must call the SAME queue walk as 2D (one attention story), so the 2D %J and the World %J can never disagree on "next". Resolve the Undo semantics in DECISIONS.md before coding it. Shell consoles never get a reply field. Shot scene rd-world-attention (mid-flight, queue strip 2 of 3, terraform card), reference 13. Then tag-ready: run /rd-gate G2.



# Integration, testing and release

## 9.1 Phase 4: integration and goldens (M455) SEQ YOU

Every lane has merged and `rd-world` is tagged. In this phase the lead stops merging and starts proving the work. `verify:visual` is hand-run and excluded from `npm run verify` (it is in the runner's `HAND_RUN` list), so goldens are refreshed **once, here**, not per lane.

- 1. **Full harness, twice.** Run `with-electron-lock.sh npm run verify` on a clean clone of `redesign/main`, then again after a reboot. That catches flakiness the `settle()` windows hide.
- 2. **All scenes.** `npm run shot` covers the 23+ existing scenes and the 14+ `rd-*` scenes, and writes every `.vs-reference.png`.
- 3. **Batch critic.** Run headless over every composite (\$4.6), in parallel. Each verdict is pasted verbatim into the ledger with any disagreement written beside it. Anything rated "does-not-read" goes back to a small fix lane (one worktree, using the same lane brief).
- 4. **Goldens on purpose.** For each changed existing scene (expect `kinds`, `kinds-dark`, `launcher`, `overview`, `attention`, `approval`, `zoomed-out(-dark)`, `palette(-dark)`, `group`, `merged` and others) and each new `rd-*` scene, which includes **the first World goldens ever**: the critic sentence goes in the ledger, **you look at the composite**, and then **you** run `UPDATE_GOLDENS=1 npm run verify:visual` in a plain terminal. The settings deny rule blocks agents from doing this.
- 5. **Docs.** Update `docs/product-rules.md` (D1 selection, D3 metrics scope, D4 focus lock, the state table), `docs/architecture-map.md` (the new modules and owners), the library-door table in `src/renderer/CLAUDE.md` (any new world files), `docs/milestone-history.md`, and the ledger's summary.

PHASE 4 · INTEGRATION (LEAD) RD\_LANE=lead, on redesign/main at rd-world

Phase 4 (M455), integration. Do not write feature code. In order:

- 1) Use verify-runner to run the full harness on a fresh clone of redesign/main (via the lock); report every red with its suite and check id.
- 2) Run npm run shot for all scenes; list every composite written.
- 3) Run the mockup-critic subagent on every rd-\* composite and every changed existing scene, in parallel; paste verdicts verbatim into the ledger under M455; produce a fix list grouped by lane.
- 4) Run rules-reviewer and world-guard over 'git diff pre-redesign...redesign/main' and add their blockers to the fix list.
- 5) Draft the docs updates listed in GUIDE.pdf §9.1 step 5 and the release notes (chord migration table from shortcuts.ts aliases). Stop and show me the fix list; I decide which fixes get a lane.

## 9.2 What proves what

| Layer                    | Covers                                                                                                                                                                                                                        | How it runs                                                                                                                                                        |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pure checks (plain node) | state-palette parity, tone mapping, attention queue order and walk, region bounds, tier hysteresis, world-space round-trip, shortcut registry and overrides, exit-code sentences, transition curves, camera math, flight path | <code>verify:rd-*</code> , <code>verify:rail</code> , <code>verify:styles</code> , <code>verify:world</code> , all concurrent, in seconds. They are G1's backbone. |

| Layer         | Covers                                                                                                                                                                       | How it runs                                                                                                                                                                                              |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Electron tier | Panel frames, approvals, ⌘Y/⌘J, the no-SIGWINCH-on-hover measurement, tier pid preservation, recovery after a tmux kill, palette behaviour, focus lock, onboarding detection | <code>verify:panels:*</code> ,<br><code>verify:canvas</code> ,<br><code>verify:xterm</code> , serial behind the lock. Watch <code>headroom.1</code> : re-pin at 90% of the watchdog, or split the suite. |
| Visual        | Reads-as-reference (critic) and pixel stability (goldens)                                                                                                                    | <code>npm run shot</code> → composites → critic; <code>verify:visual</code> at G5                                                                                                                        |
| Packaged      | The World in a production build (D6), the lazy chunk under <code>file://</code> , native node-pty, tmux persistence across quit                                              | <code>npm run verify:packaged</code> , then a smoke test of the .app by hand                                                                                                                             |

## 9.3 Performance and robustness checks

- **WebGL context budget.** Eight live xterm-webgl terminals, the World (with bloom) and a minimap must not hit the browser's ~16-context cliff. Test: open 8 pinned terminals, toggle the World 20 times, and check that no context is lost. Then force context loss and check that WorldFlat takes over.
- **First chunk.** `npm run build`, then check that the entry chunk statically imports no three or postprocessing (`orch-zoom.3`, `world.door.*`). Record the chunk sizes before and after in the ledger.
- **Frame time.** The canvas at 40 panels in Plan tier, and the World at 20 agents. Use `world-perf.ts` budgets and record the numbers. Semantic tiers must make zoomed-out canvases *cheaper*, because cards replace live terminals.
- **Input safety.** A stray Enter in the World or in Sessions never reaches a shell prompt. Keystrokes are disabled while a session is paused. While focus lock is on, ⌘K reaches the terminal.

## 9.4 Manual QA checklist YOU

Use a real Mac, the real `claude` and `codex` CLIs, tmux, and a real repo. Run part A at the end of Phase 2 and part B at the end of Phase 3. Run all three parts at release.

### A · CANVAS (01–09)

- ☐ Quit with 5 sessions, relaunch: the restore lines count truthfully. Hold `⌘`: the rest come up asleep, and their tmux panes are still alive (`tmux ls`).
- ☐ Fresh profile: onboarding finds claude and codex, a missing gemini offers a copyable install command, and nothing spawns until Continue / Start task.
- ☐ Empty canvas: one primary verb, no zeros, and double-click places a terminal at the cursor.
- ☐ Make codex ask for an edit: amber edge and glow, and it is the only glowing thing. The pill, dock badge, Sessions count, pip and minimap all say the same number. ⌘Y allows it.
- ☐ Hover a terminal header 20 times: the agent never redraws (no SIGWINCH).

### B · WORLD (10–14)

- ☐ ⌘⇧W from a zoomed spot: the plan tilts into the floor at the same place, and the robots near the target pop first. Esc mid-move reverses smoothly. ⌘⇧W back lands 2D on the same spot.
- ☐ Reduced motion (System Settings): a 120 ms cross-fade and no flights, in both views.
- ☐ Terraces match the 2D layout. Dragging a terrace moves the region in 2D, and one ⌘Z restores it.
- ☐ Colours: cyan working, amber waiting (with beacon), green done, red failed (slumped, still at its desk), slate idle. No candy-coloured shells.
- ☐ Click a waiting robot: glide, Follow, breadcrumb, sheet. Approve ⌘Y uses the same path as 2D (it shows in the 2D inspector history).

- Drag a port to empty canvas, pick Test watcher: it starts when the source exits, with its summary.
- Marquee 3 panels → ⌘G makes a task, and ⌘Z undoes it in one step. ⌘⇧T and ⌘⇧T both tidy.
- ⌘2/⌘3/⌘1: Plan cards, then Map territories, then live terminals. No session restarted (pids unchanged). A terminal focused before ⌘2 no longer takes keys.
- ⌘K with focus lock off opens the palette. With ⌘⇧L on, claude clears its screen.
- Sessions (⌘⇧S): approve from a card, bulk-pause 3, and reply to an agent with explicit Send. The shell prompt card has no answer field.
- Settings > Keyboard: re-record a chord, and the menu shows it. A bare key is refused. A duplicate is flagged.
- `tmux kill-server` mid-run: banner, paused panels, auto-retry, honest end states. `kill -9` an agent: the exit 137 card restarts with the last prompt.
- Wi-Fi off: the toast appears, GitHub shows offline · cached, and local terminals keep running.
- ⌘J flies the amber path to the next agent, and the strip reads "2 of 3". The terraform console offers only Open in Canvas, Snooze and Next.
- Undo after approve behaves exactly as DECISIONS.md says.
- Map tier: ⌘O centres the room. Replay a past moment: verbs are disabled with their reason, and Live returns.
- Away for 40+ minutes: the "While you were away" tour plays real events.
- Disable WebGL (or force the probe off): the flat room has the same words and doors.
- Open in Canvas ⌘⇧J / double-click: 2D lands on the panel with it selected.

## C · CROSS-CUTTING

- Light theme on every screen: state colours still separable (contrast check 11 values).
- Compact (1000px) and wide (1800px) breakpoints: the inspector, Sessions detail and focus sheet behave as drawers or columns.
- Keyboard only: every screen is reachable, and Tab from ⌘Esc walks the chrome.
- VoiceOver announces new "needs you" items once (`useAttentionAnnouncer`) and doesn't repeat the pill's toast.
- Packaged .app: the World loads under `file://`, and quit-and-reopen keeps sessions.

## 9.5 Phase 5: release (M456) SEQ

1. Fixes from Phase 4 have merged. The full harness is green twice. Golden sentences are signed (ledger sentences present).
2. Bump `package.json` to **6.0.0** (D8). Then run `verify:package` / `verify:meta` (version mentions in docs may be pinned) and update the CLAUDE.md line "This is 5.0 (M192)".
3. Write the release notes: the redesign in one line, the five state colours, the new chords, the alias table (⌘⇧T → ⌘⇧T), focus lock, Sessions, and the World changes. Mockups 04, 07 and 11 make good screenshots.
4. `npm run verify:packaged`, then `npm run package` (`electron-builder.config.cjs`). Smoke test the .app with part C of the QA checklist.
5. Open a PR from `redesign/main` into `main`. CI runs `npm run verify` on macos-latest. Merge with `--no-ff`, tag `v6.0.0`, and push the tags.
6. Clean up: `git worktree remove ../tc-rd-*`, `git worktree prune`, and delete the merged `rd/*` branches. Close the ledger with a summary and links to every milestone commit.



Phase 5 (M456), release 6.0.0. Confirm Phase 4's fix list is closed and the ledger has a critic sentence for every changed golden (list any missing and stop). Bump the version, update CLAUDE.md's version line and any doc that verify:meta / verify:package pins, write RELEASE-NOTES for 6.0.0 from the ledger (include the chord alias table generated from src/shared/shortcuts.ts), run verify:packaged and npm run package via the lock, and open the PR redesign/main → main with gh. Do not merge or tag: I will, after the smoke test.

# Risks and gotchas

| Risk                                      | Why it bites in this repo                                                                                                                                                                         | Mitigation (and where it's handled)                                                                                                                                           |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ⌘K clash                                  | Claude Code and macOS terminals use ⌘K to clear the screen. The palette rule reads <code>event.code</code> , so it fires whatever is focused.                                                     | D4: the canvas owns ⌘K except under focus lock (⌘⇧L). Settings > Keyboard shows the conflict banner (O8). Checks cover both paths (F3, L-C).                                  |
| Colour-meaning conflict, World vs. canvas | The World paints WORKING green and QUIET red (M415). The canvas paints idle green and working blue. Robot candy tints include green, red, orange and teal, so a shell colour can read as a state. | One table (F1), one hex source ( <code>state-palette.ts</code> ), parity and literal checks, the identity-hue ΔE check (W0), and a critic step that checks state colour last. |
| Cyan means two things                     | The concept makes cyan both brand/selection and "working". M63's rule kept them apart.                                                                                                            | D1: a distinct selection ring and halo. Record the rule change. The critic checks for a selected working panel in <code>rd-f1-tones</code> .                                  |
| WebGL fallback and budget                 | xterm-webgl uses up to LIVE_BUDGET 8 contexts, and browsers drop contexts near 16. The World adds a context plus a bloom composer. Machines without WebGL exist, and so does context loss.        | <code>webgl-probe.ts</code> before mount. WorldFlat has the same words and doors. Context loss falls back to flat (W6). The budget test is in §9.3. Never raise LIVE_BUDGET.  |
| three in the first chunk                  | One static import adds about 2.2 MB to the first chunk, and nothing in the UI shows it.                                                                                                           | <code>world-guard</code> , <code>world.door.*</code> , <code>orch-zoom.3</code> , and only one <code>lazy()</code> in WorldStage.                                             |
| SIGWINCH on hover                         | A header box that collapses on hover refits xterm and interrupts the agent, with no error.                                                                                                        | Chromeless chrome is absolutely positioned. L-B measures the well and allows Opx change.                                                                                      |
| Renaming DOM hooks                        | About 200 checks select <code>.panel__*</code> / <code>*-node__*</code> / <code>.shell__*</code> .                                                                                                | Restyle, don't rename (rules-reviewer). New classes are added beside the old ones.                                                                                            |
| Semantic zoom vs. live tiering            | LIVE_MIN_SCALE is 0.5, Work starts at 0.70, and a focused panel is always live, so keys could land in a card.                                                                                     | Tiers feed <code>cardIds</code> , entering Plan/Map releases focus, and pid-preservation checks run (F2, L-C).                                                                |
| Parallel Electron runs collide            | Fixed fixture directory names and one verify tmux socket. <code>TC_VERIFY_ELECTRON_JOBS&gt;1</code> is measured unreliable.                                                                       | <code>with-electron-lock.sh</code> on every Electron-tier run. At most four lanes per wave.                                                                                   |
| Native module per worktree                | node-pty is rebuilt against Electron at postinstall. A shared or symlinked <code>node_modules</code> breaks.                                                                                      | <code>lane.sh</code> runs <code>npm ci</code> + the Electron install in each worktree. Budget the disk space.                                                                 |
| Wrong worktree base                       | <code>claude --worktree</code> and <code>isolation:worktree</code> branch from the default branch ( <code>main</code> ) unless <code>worktree.baseRef</code> is <code>"head"</code> .             | Lanes use explicit <code>git worktree add ... rd-&lt;tag&gt;</code> . Settings set <code>baseRef: "head"</code> for subagent worktrees.                                       |

| Risk                                     | Why it bites in this repo                                                                                                                         | Mitigation (and where it's handled)                                                                                                                                       |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>verify:meta tripwires</b>             | The ledger link in CLAUDE.md, scoped check ids, counts written in prose, the IPC channel list in CLAUDE.md, and suites derived from package.json. | Phase 0 pre-registers suites and links the ledger. Lanes use scoped ids. Main-process lanes update the channel list (L-A, L-F).                                           |
| <b>Blind golden re-baseline</b>          | A golden refreshed without anyone looking hides a regression for good.                                                                            | A deny rule on <code>UPDATE_GOLDENS=1</code> for agents. Critic sentences go in the ledger first. You run the update (Phase 4).                                           |
| <b>Stray Enter runs a command</b>        | Reply boxes in Sessions and the World could reach a shell prompt (terraform apply).                                                               | Shell prompts are answered only in the terminal. Replies use paste semantics with an explicit Send (L-D, W4, W5).                                                         |
| <b>"Undo" after Approve</b>              | An agent that already received "y" can't be untold.                                                                                               | Decide between a short visible hold and revert-only undo in DECISIONS.md before W5 starts.                                                                                |
| <b>Metrics rule vs. mockups</b>          | Cost and run columns, and "\$3.12 today" in the title bar, break <code>metrics.1</code> / <code>hud.2</code> as written.                          | D3: scope the rule. Drop the \$ from the title bar.                                                                                                                       |
| <b>Mockup sample copy</b>                | "Steward", SW-412 and the dollar figures are demo data.                                                                                           | Fixtures only. The critic ignores sample copy (it is already on product-rules' "not copied" list).                                                                        |
| <b>1440×900 vs. 1440×865</b>             | The mockups and the shot window differ by 35 px of height.                                                                                        | The critic judges reading, not pixels, and pixel budgets apply only to goldens.                                                                                           |
| <b>Review tab has no mockup</b>          | The nav adds Review, but O1–14 never show it.                                                                                                     | F3 hosts the existing across-worktree review (the <code>across</code> scene) in the Review view. Redesigning Review is out of scope and is logged in the backlog.         |
| <b>Lane drift and context bloat</b>      | Long lane sessions forget their constraints.                                                                                                      | The SessionStart brief hook, subagents for heavy reads, <code>/compact</code> between milestones, and the ledger (not memory) as the state, which is the repo's own rule. |
| <b><code>ELECTRON_RUN_AS_NODE</code></b> | VS Code terminals export it, and Electron scripts then fail at <code>whenReady</code> .                                                           | The lock script unsets it. Run lanes from tmux, not from the VS Code terminal.                                                                                            |

# Lane cheat sheet

| Lane | Mockups      | Milestone | Wave · base         | Key files owned                                                                   | Shot scenes                        |
|------|--------------|-----------|---------------------|-----------------------------------------------------------------------------------|------------------------------------|
| F1   | (04, 07, 11) | M436      | P1 · rd-p0          | styles.css tokens, panel-state.ts, state-palette.ts                               | rd-f1-tones                        |
| F2   | (04, 06, 11) | M437      | P1 · rd-p0          | attention-queue, task-regions, zoom-tier, world-space, fixture                    | —                                  |
| F3   | 04, 08       | M438      | P1 · rd-p0          | TopBar, useShellChrome, shortcuts.ts, CommandPill, menu.ts                        | rd-f3-titlebar, rd-f3-pill         |
| L-A  | 01, 02, 03   | M439–M441 | 2a · rd-foundations | StartupSplash, onboarding/*, EmptyState, Launcher, env-report                     | rd-splash, rd-onboarding, rd-empty |
| L-B  | 04, 05       | M442–M443 | 2a · rd-foundations | <b>Canvas.tsx</b> , panels/*, TaskClusterLayer, Inspector, links, guides, marquee | rd-workspace, rd-arrange           |
| L-D  | 07           | M445      | 2a · rd-foundations | sessions/* (new)                                                                  | rd-sessions                        |
| L-E  | 08           | M446      | 2a · rd-foundations | settings/* (new), shortcut-overrides.ts, settings-schema                          | rd-settings-keys                   |
| L-C  | 06           | M444      | 2b · rd-wave2a      | <b>Canvas.tsx</b> (TierLayer slot), palette/*, CanvasHud, MinimapOverlay, flight  | rd-plan-palette, rd-map            |
| L-F  | 09           | M447      | 2b · rd-wave2a      | Resume/JobRecovery banners, session-backend, exit-explain, frame variants         | rd-recovery                        |
| W0   | 11–13        | M448      | 3a · rd-canvas      | world-palette, WorldRobot, bloom, roster                                          | rd-world-night                     |
| W1   | 10           | M449      | 3b · rd-w0          | world-transition, WorldStage, WorldLens, plan-floor                               | rd-world-transition(-rm)           |
| W2   | 11           | M450      | 3b · rd-w0          | <b>WorldView.tsx</b> , WorldStructure, WorldOffice, WorldCard, WorldChrome        | rd-world-room                      |
| W3   | 14           | M451      | 3b · rd-w0          | world-camera, WorldTime, replay, away, peers                                      | rd-world-overview                  |
| W4   | 12           | M452      | 3c · rd-w3b         | <b>WorldView.tsx</b> , world-select, WorldFocusSheet                              | rd-world-focus                     |
| W6   | (07, 11)     | M454      | 3c · rd-w3b         | WorldFlat, context-loss, View/Show in World doors                                 | rd-world-flat                      |
| W5   | 13           | M453      | 3d · rd-w3c         | <b>WorldView.tsx</b> , world-flight, WorldQueueStrip                              | rd-world-attention                 |

Tags in order: `pre-redesign` → `rd-p0` → `rd-foundations` → `rd-wave2a` → `rd-canvas` → `rd-w0` → `rd-w3b` → `rd-w3c` → `rd-world` → `v6.0.0`. Milestones follow the repo's build log numbering (the last existing is M432–M434). Renumber them if anything lands on `main` first.

# Claude Code features this plan relies on

Checked against Anthropic's Claude Code docs ([code.claude.com/docs](https://code.claude.com/docs): *sub-agents*, *worktrees*, *agents*, *hooks*, *settings-example*) on October 5, 2026. Run `claude update` first. Some subagent fields need recent versions.

| Feature                                            | How this plan uses it                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CLAUDE.md project memory (nested)                  | The root index plus the existing nested files. Redesign section in §4.1. Subagents load CLAUDE.md, except Explore/Plan and agents with <code>omitClaudeMd: true</code> (the critic).                                                                                                                                                                                                                                                                                                  |
| Custom subagents, <code>.claude/agents/*.md</code> | Six agents. Frontmatter: <code>name</code> , <code>description</code> , <code>tools</code> , <code>model</code> , plus <code>omitClaudeMd</code> , <code>isolation</code> , <code>permissionMode</code> , <code>hooks</code> , <code>memory</code> , <code>effort</code> where useful. Invoke them by name, with @-mention, or with <code>claude --agent</code> . The Task tool is now called <code>Agent</code> (since v2.1.63), and <code>Task(...)</code> still works as an alias. |
| Parallel subagents / background                    | Parallel lb-scout reads and parallel critics. Subagents run in the background by default in interactive sessions. Check them with <code>/tasks</code> .                                                                                                                                                                                                                                                                                                                               |
| Worktrees                                          | Lanes use explicit <code>git worktree add</code> . <code>claude --worktree &lt;name&gt; / -w</code> and <code>isolation: worktree</code> are available for disposable work. <code>worktree.baseRef: "head"</code> keeps them on the current branch.                                                                                                                                                                                                                                   |
| Plan mode                                          | Every lane starts with <code>claude --permission-mode plan</code> (or Shift+Tab). You approve the plan before any edit.                                                                                                                                                                                                                                                                                                                                                               |
| Slash commands                                     | <code>.claude/commands/*.md</code> with <code>\$ARGUMENTS</code> (now treated as skills; <code>.claude/skills/&lt;name&gt;/SKILL.md</code> is equivalent).                                                                                                                                                                                                                                                                                                                            |
| Hooks                                              | <code>SessionStart</code> (lane brief into context), <code>PreToolUse</code> on Edit/Write (ownership guard; exit 2 blocks and returns stderr), <code>PostToolUse</code> (targeted quick checks), <code>Stop</code> (typecheck; exit 2 keeps Claude working; <code>stop_hook_active</code> prevents loops), <code>Notification</code> (macOS banner when a lane needs you).                                                                                                           |
| Permission rules                                   | <code>allow / ask / deny</code> in <code>.claude/settings.json</code> . Force-push, <code>reset --hard</code> and <code>UPDATE_GOLDENS=1</code> are denied to agents.                                                                                                                                                                                                                                                                                                                 |
| Headless mode                                      | <code>claude -p "" --output-format json --allowedTools ...</code> for batch critic runs and digests.                                                                                                                                                                                                                                                                                                                                                                                  |
| Optional                                           | Agent teams, agent view and the bundled <code>/batch</code> skill exist. This plan uses <code>/batch</code> only for mechanical sweeps inside one lane.                                                                                                                                                                                                                                                                                                                               |

IN ONE PARAGRAPH

Lock the decisions. Build the run kit and the pre-cut seams. Land three foundation lanes and freeze them. Fan out to four, then two, canvas lanes. Then rebuild the World in four waves on the same foundations. Every lane starts in plan mode, edits only the files it owns, proves itself with pure checks first, and is judged against its mockup by a fresh-context critic. A person signs every golden.